Спецтрек ФСП · Лидеры цифровой трансформации 2026

fsp-talent

Платформа подбора IT-специалистов с верифицированным профилем достижений участника ФСП

Сопроводительная документация к решению

Команда: [Название команды]

Стенд: https://159-194-200-243.sslip.io

Swagger API: https://159-194-200-243.sslip.io/api/docs

Репозиторий: https://github.com/Vadik145-beep/fsp-talent

Версия документа: 7 октября 2026

Документ собран из разделов docs/solution репозитория (python docs/solution/tools/build_docs.py). Разделы 1–8 соответствуют пунктам ТЗ 3.4.

Оглавление

fsp-talent — сопроводительная документация к решению 6
Решение в пяти пунктах 6
Оглавление (пункты ТЗ 3.4) 6
Соответствие требованиям ТЗ 6
Статус реализации (на 07.10.2026) 7
Документ одним файлом (DOCX и PDF) 7
1. Функциональная и компонентная архитектура 9
1.1. Что делает система 9
1.2. Роли и кабинеты 9
1.3. Контейнеры (развёртывание) 9
1.4. Слои и компоненты бэкенда 10
1.5. Фронтенд 12
1.6. Карта экранов 12
1.7. Основные потоки 13
1.8. Безопасность и 152-ФЗ 14
1.9. Масштабирование и интеграция 14
2. Механика тестирования и устойчивость к распространению заданий 15
2.1. Проблема и наш ответ 15
2.2. Путь кандидата 15
2.3. Банк параметрических шаблонов 15
2.4. Модель IRT и оценка уровня 16
2.5. Адаптивный тест (CAT) 16
2.6. Грейд и правила частоты 17
2.7. Детектор утечек 17
2.8. Калибровка и жизненный цикл шаблона 17
2.9. Задания и тесты из текста вакансии 18
2.10. Почему механика устойчива к распространению заданий 18
2.11. Почему не «одна нейросеть генерирует задания каждому» 18
2.12. Результаты проверки механики 18
2.13. Ограничения и что дальше 19
3. Механика подбора и логика категоризации 20
3.1. Идея: работодатель приходит сам 20

3.2. Категоризация кандидата....20
3.3. Потребность работодателя....20
3.4. Рекомендованные категории....21
3.5. Ранжирование внутри категории....21
3.6. Объяснимость: «почему в подборке»....21
3.7. Уточнение запроса без потери подборки....22
3.8. Обезличенный профиль....22
3.9. Приглашение: выход на контакт....22
3.10. Рыночная подсказка....23
3.11. Отклик на вакансию — дополнительный сценарий....24
3.12. Регулярные задания и свежесть профиля....26
3.13. Защита от недобросовестных работодателей и интеграция с ATS....26
3.14. Качество подбора....27
4. Процедура валидации и результаты....28
4.1. Принципы....28
4.2. Что проверяем и чем — по критериям ТЗ 6.1 и 6.2....28
4.3. Данные....28
4.4. Результаты: механика тестирования....29
4.5. Результаты: подбор....32
4.6. Результаты: разбор текстов....34
4.7. Как воспроизвести и проверить самим....34
4.8. Ограничения валидации....34
5. Схема интеграции с реестром ФСП....36
5.1. Какие данные получаем и зачем....36
5.2. Вход и привязка через ФСП ID (OIDC)....36
5.3. Контракт API реестра (наш мок)....37
5.4. Обновление данных....38
5.5. Если истории ФСП нет....38
5.6. Что показываем работодателю....39
5.7. ФСП-скор....39
5.8. Безопасность и согласие....39
5.9. Учебный стенд ФСП ID....39
5.10. Что нужно от ФСП для реальной интеграции....40
6. Описание API....41

6.1. Где смотреть....41
6.2. Общие правила....41
6.3. Группы методов....41
6.4. Авторизация....42
6.5. Формат ошибок....43
6.6. Примеры....44
6.7. API мока реестра ФСП....44
Приложение к разделу 6. Эндпоинты API по ролям....45
Служебное (system)....45
Регистрация, вход, ФСП ID (auth)....45
Согласия 152-ФЗ (users)....46
Справочники (catalog)....46
Кабинет соискателя и банк кандидатов (candidates)....46
Компания рекрутера (companies)....46
Опрос, адаптивный тест, грейд (testing)....47
Достижения ФСП (fsp)....47
Потребности и подборка (matching)....47
Приглашения и выход на контакт (invitations)....48
Вакансии и отклики (vacancies)....48
Регулярные задания (tasks)....48
Кабинет администратора, жалобы (admin)....49
Валидация и метрики (eval)....50
Коды ошибок....50
7. Библиотеки и компоненты с версиями....52
Платформа....52
Библиотеки....52
Образы Docker....54
8. Сборка, развёртывание и локальный запуск....55
8.1. Что нужно....55
8.2. Локальный запуск полного стека....55
8.3. Переменные окружения....56
8.4. Стенд на сервере (один сервер, Caddy + HTTPS)....57
8.5. Разработка....58
8.6. Периодические задачи (cron)....58

8.7. Частые проблемы.....59

fsp-talent — сопроводительная документация к решению

Платформа подбора IT-специалистов с верифицированным профилем достижений участника ФСП. Спецтрек ФСП, ЛЦТ 2026.

Стенд	https://159-194-200-243.sslip.io
Swagger	https://159-194-200-243.sslip.io/api/docs
Репозиторий	https://github.com/Vadik145-beep/fsp-talent
Лицензия	MIT

Демо-учётки (пароль у всех `demo1234`): соискатель `candidate@example.com` (Backend · Middle, с ФСП ID), соискатель без истории ФСП `candidate.nofsp@example.com` (Frontend · Junior), рекрутер `recruiter@example.com` (компания «Северный код»), администратор `admin@example.com`.

Решение в пяти пунктах

1. **Обратная механика подбора.** Работодатель описывает потребность, получает подборку по категориям (специализация × грейд) с объяснением «почему в подборке» и сам приглашает кандидата с обязательной зарплатной вилкой. Контакты открываются только после принятия приглашения. Квота приглашений, запрет вилки ниже ожиданий кандидата, отказ с причиной, рыночная подсказка «обычно принимают от X ₽».
2. **Категорию определяет тест, а не резюме.** Опрос → выбор грейда → адаптивный тест (IRT, CAT) на банке из 192 параметрических шаблонов. Грейд принудительно не понижается, смена — не чаще раза в 90 дней.
3. **Тест устойчив к утечкам.** У каждого кандидата свой вариант задачи, сложность шаблонов измерена и калибруется по ответам, детектор утечек автоматически выводит «слитые» задачи. Нейросеть только предлагает черновики заданий по тексту вакансии — оценку она не делает.
4. **Подтверждённый профиль.** Достижения ФСП приходят из реестра через привязку ФСП ID (Keycloak, OIDC), превращаются в бейджи и ФСП-скор и входят в ранжирование. Кандидат без истории ФСП не штрафуется.
5. **Проверено на данных с известным ответом.** На 1200 синтетических кандидатах тест восстанавливает уровень с корреляцией 0,94 и ставит верную категорию в 78,8% случаев (ошибка больше чем на грейд — 0%; заявленный грейд верен лишь в 64,9%). Подбор даёт precision@10 0,486 против 0,253 у поиска по ключевым словам резюме. Параметризация снижает выгоду от утечки 20% банка с 39% до 12,7% завышенных грейдов. Жюри может загрузить свои пары и получить метрики.

Оглавление (пункты ТЗ 3.4)

№	Раздел ТЗ 3.4	Документ
1	Функциональная и компонентная архитектура	01_architecture.md
2	Механика тестирования и обоснование устойчивости к распространению заданий	02_testing.md
3	Механика подбора и логика категоризации	03_matching.md
4	Процедура валидации и её результаты	04_validation.md
5	Схема интеграции с реестром ФСП	05_fsp_integration.md
6	Описание API	06_api.md, приложение api_endpoints.md
7	Библиотеки и компоненты с версиями	07_libraries.md
8	Сборка, развёртывание и локальный запуск	08_deployment.md

Подробные технические материалы, на которые ссылаются разделы: `docs/testing/engine.md` (движок теста и симуляция), `docs/testing/bank/` (банк задач по специализациям), `docs/testing/survey.md` (опрос), `docs/integration/fsp.md` (интеграция с ФСП), `docs/architecture/er.md` (схема данных), `docs/validation/` (отчёты валидации), `docs/api/openapi.json` (схема API).

Соответствие требованиям ТЗ

Требование	Где реализовано	Раздел
Категория по опросу и тесту, не по резюме [2, 2.1]	`testing`, `matching`	2, 3.2
Механика тестирования, устойчивая к распространению [2]	`testing/engine`	2
Тест ниже/выше, ограничение частоты смены грейда [2]	`decide`, `can_attempt`, 90 / 30 дней	2.6
Задания из описаний вакансий [2]	`admin/vacancy-tests`,	2.9

Требование	Где реализовано	Раздел
	`nlp/vacancy_tests`	
Регулярные задания, свежесть профиля [2, 2.1]	`tasks`, cron `python -m app.tasks`	3.12
Выход на контакт по инициативе работодателя [2]	`invitations`	3.9
Ранжирование внутри категории с объяснимостью [2.1]	`matching/scoring.py`	3.5, 3.6
Фильтры без потери подборки [2.1]	подборка и банк кандидатов	3.7
Зарплата «от–до» обязательна в вакансии и приглашении [2.2 п.1]	схемы и CHECK в БД	3.9
Контакты только после принятия или отклика [2.2 п.2]	`to_card` / `to_contacts`	3.8
Статусы приглашений и откликов видны обеим сторонам [2.2 п.3]	`invitations`, `vacancies`	3.9, 3.11
Регистрация по почте с подтверждением [2.2 п.4]	`auth`	6.4
Профиль из PDF с автораспознаванием, PDF-профиль [2.1]	`candidates`, `nlp`	4.6
Связь с ФСП ID, совместимость с Keycloak; кандидат без истории [2, 3.5]	`fsp`, Keycloak realm `fsp`	5
Собственная процедура валидации и результаты [2]	`eval`, симуляции	4
Frontend / Backend / СУБД, Docker, OpenAPI [3.5]	`frontend`, `backend`, PostgreSQL, compose	1, 6, 8
152-ФЗ: согласия, права по ролям, хранение паролей и токенов [3.5]	`users`, `core/security`, `auth`	1.8
Открытый код, открытые лицензии [3.5]	MIT, зависимости MIT/BSD/Apache	7
Защита от фиктивных вакансий, интеграция с ATS [2]	жалобы, блокировка и скоринг компаний, API-токены, выгрузка и вебхуки ATS; `docs/concepts/antifraud_ats.md`	3.13
Адаптивная вёрстка [3.5]	фронтенд	1.5

Статус реализации (на 07.10.2026)

Часть	Состояние
Регистрация, вход, подтверждение почты, сброс пароля, согласия	на БД, работает
Профиль, загрузка PDF-резюме, PDF-профиль, приватность	на БД, работает
Опрос, адаптивный тест, грейд и кулдауны, банк 192 шаблона	на БД, работает
Детектор утечек, калибровка, тесты из вакансий (админ)	на БД, работает
ФСП ID (Keycloak), реестр, ФСП-скор, вебхук	работает (реестр — наш мок)
Потребности, подборка с объяснением, банк кандидатов	на БД, работает
Приглашения: статусы, квота, подсказка, контакты, письма	на БД, работает
Вакансии и отклики: лента с фильтрами, отклик с контактами, статусы, письма	на БД, работает
Регулярные задания: раздача, ответ, оценка 0–3, свежесть профиля	на БД, работает; периодические работы — cron или запрос админа (8.6)
Дашборд админа, жалобы, блокировка и скоринг компаний	на БД, работает
ATS: API-токены, выгрузка, вебхуки с подписью и повторами	работает; подтверждение домена и ИНН, пробный режим — концепция
Валидация: синтетика с эталоном, отчёт, графики, проверка CSV жюри	работает, `make validate`
Нагрузочный замер	не проводился; ориентир Т3 — десятки одновременных пользователей (1.9)

Банк задач в админке показывает 222 записи: 192 рабочих шаблона движка и 30 демо-записей из фикстур (они иллюстрируют экраны, в тест не попадают).

Документ одним файлом (DOCX и PDF)

Готовые файлы: `fsp-talent-documentation.pdf` и `fsp-talent-documentation.docx` — все разделы в порядке оглавления, с титульным листом, оглавлением, диаграммами и графиками валидации.

Разделы написаны в Markdown с диаграммами mermaid. Сборка (нужны Node.js для mermaid-cli и Docker для pandoc и LibreOffice):

```
python docs/solution/tools/gen_api_table.py      # таблица раздела 6 из openapi.json
python docs/solution/tools/gen_libraries.py      # раздел 7 (при поднятом docker compose)
python docs/solution/tools/build_docs.py         # → docs/solution/fsp-talent-documentation.
{docx,pdf}
```

`build_docs.py` склеивает разделы (`build_bundle.py --render-mermaid`: диаграммы → PNG, ссылки между разделами → якоря, остальные ссылки → GitHub), конвертирует в DOCX через pandoc и обновляет оглавление и экспортирует PDF в LibreOffice.

1. Функциональная и компонентная архитектура

1.1. Что делает система

fsp-talent — веб-платформа подбора IT-специалистов с **обратной механикой**: не кандидат откликается на вакансии, а работодатель находит нужную **категорию** кандидатов и сам приглашает конкретного человека, сразу указывая зарплатную вилку.

Категория = **специализация × грейд** (6 × 4 = 24 категории, например «Backend · Middle»). Её определяет не резюме, а обязательный путь кандидата: опрос по специализации → выбор предполагаемого грейда → адаптивный тест. Внутри категории кандидаты ранжируются по силе подтверждённого профиля: результат теста, стек, достижения ФСП, свежесть профиля, зарплата, формат работы — с объяснением «почему в подборке».

1.2. Роли и кабинеты

Роль	Код	Кто это	Главное в кабинете
Соискатель	`candidate`	участник рынка, ищет работу	профиль и PDF-резюме, опрос и тест, категория и грейд, приглашения (принять / отклонить с причиной), вакансии и отклики, регулярные задания, достижения ФСП, приватность и согласия
Рекрутер	`recruiter`	представитель компании	профиль компании, потребности (форма или свободный текст), подборка по категориям с объяснением, банк кандидатов с фильтрами, приглашения со статусами и квотой, вакансии и отклики, регулярные задания
Администратор	`admin`	оператор платформы (ФСП)	дашборд, пользователи и компании (блокировка, скоринг надёжности), справочники и настройки (пороги θ, кулдауны, квота, веса ранжирования), банк задач (метрики, утечки, калибровка), тесты из вакансий, валидация, жалобы, журнал действий

Роль администратора в ТЗ не названа — она нужна для банка задач, детектора утечек, настроек ранжирования и метрик валидации.

Права проверяются на бэкенде в каждом методе (зависимости `CandidateUser`, `RecruiterUser`, `AdminUser` в `backend/app/core/security.py`): без входа — `401`, чужая роль — `403`, чужой объект — `404` (кандидат видит только свои приглашения и отклики, рекрутер — только записи своей компании). Таблица методов по ролям — приложение к разделу 6.

1.3. Контейнеры (развёртывание)

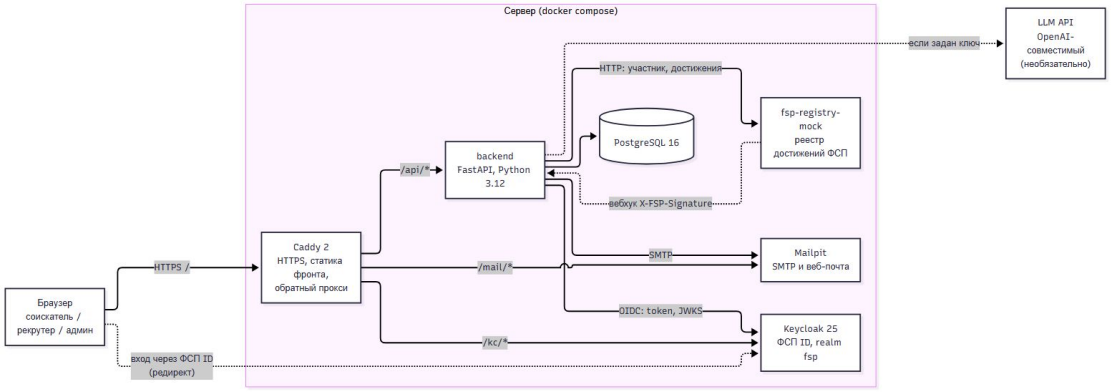


Диаграмма 1

Контейнер	Образ	Назначение	Порт в dev-стеке
`frontend`	собственный (node:22-alpine)	React SPA; в dev — Vite, на стенде — статика за Caddy	15173
`backend`	собственный (python:3.12-slim)	REST API, бизнес-логика, движок теста	18000

Контейнер	Образ	Назначение	Порт в dev-стеке
`postgres`	postgres:16-alpine	хранение всех данных	15432
`keycloak`	quay.io/keycloak/keycloak:25.0	ФСП ID: вход и привязка профиля (OIDC + PKCE)	18180
`fsp-registry-mock`	собственный (python:3.12-slim)	наш вариант API реестра достижений ФСП	18100
`mailpit`	axllent/mailpit	письма подтверждения почты, приглашений, сброса пароля	18025 (UI), 11025 (SMTP)
`caddy` (только стенд)	caddy:2-alpine	HTTPS по Let's Encrypt, один адрес для всего	80, 443

Суммарный лимит памяти в `infra/stand/compose.yml` — около 2,4 ГБ (сервер 2 vCPU / 4 ГБ): backend 768 МБ, Keycloak 900 МБ, PostgreSQL 384 МБ, мок 192 МБ, Mailpit 128 МБ.

Без внешних сервисов система работает полностью: LLM необязателен (разбор текстов идёт на правилах и словарях), реестр ФСП и Keycloak — свои контейнеры. Если Keycloak или реестр недоступны, основной вход по почте и весь подбор работают; достижения показываются из кэша.

1.4. Слои и компоненты бэкенда

Бэкенд — **модульный монолит** с явными слоями. Каждый модуль `backend/app/<модуль>/` устроен одинаково: `router.py` (HTTP, схемы, коды ошибок) → `service.py` (правила) → `repository.py` (SQL через SQLAlchemy) → `models.py` (таблицы), `schemas.py` (Pydantic). Модули общаются через сервисы, а не через чужие таблицы напрямую. Движок теста — чистая библиотека без БД и веба, его можно вынести в отдельный сервис без переделки.

Основные связи между модулями (стрелка — «использует»):

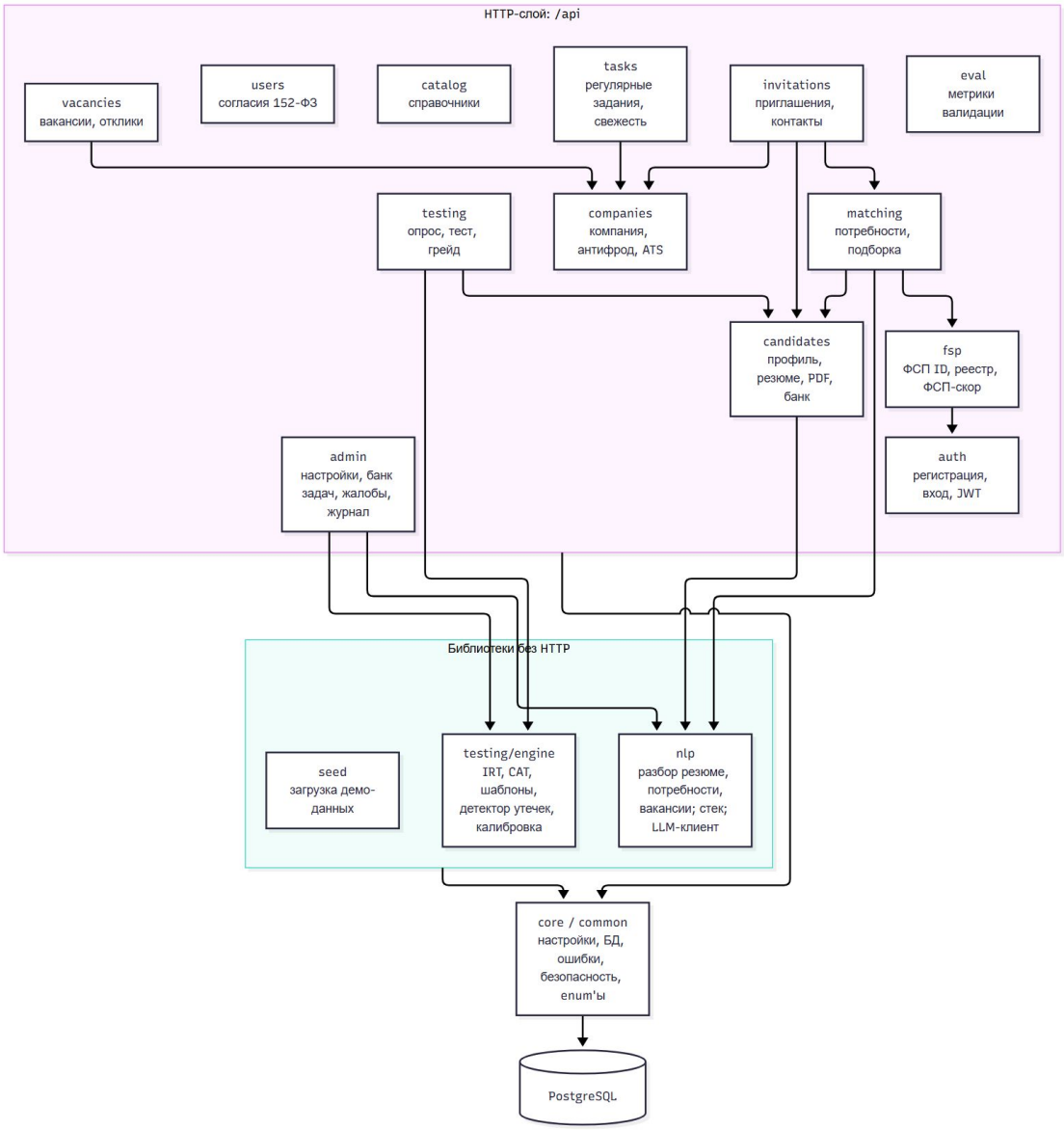


Диаграмма 2

Модуль	Что делает	Ключевые таблицы
`auth`	регистрация по почте с подтверждением, вход, JWT access + refresh-cookie, сброс пароля, лимит попыток входа	`users`, `email_tokens`, `refresh_sessions`, `auth_throttle_events`
`users`	согласия на обработку ПДн и на публикацию профиля (версия текста, даты)	`consents`
`catalog`	специализации, грейды, категории, темы теста, стек, причины отказа, настройки платформы	`specializations`, `grades`, `categories`, `topics`, `skills`, `platform_settings`
`candidates`	профиль, загрузка PDF-резюме с разбором, PDF-профиль, приватность, пауза поиска, банк кандидатов (обезличенно)	`candidate_profiles`, `candidate_skills`, `resumes`
`companies`	профиль компании, рекрутеры, код приглашения рекрутеров, дашборд рекрутера; блокировка компании (антифрод); вложенный модуль `integrations` — API-токены, выгрузка и вебхуки ATS	`companies`, `recruiters`, `company_api_tokens`, `webhook_endpoints`, `webhook_deliveries`
`testing`	опрос, сессии адаптивного теста, ответы, итог и история грейда, синхронизация банка	`items`, `test_sessions`, `test_answers`, `grade_history`
`testing/engine`	2PL IRT, EAP-оценка θ, CAT, 192 параметрических шаблона, решение по грейду, кулдауны, детектор утечек, калибровка, симуляция	— (чистые функции)
`fsp`	вход и привязка через ФСП ID (OIDC + PKCE), клиент реестра, кэш достижений, ФСП-скор и бейджи,	`fsp_links`, `fsp_achievements`

Модуль	Что делает	Ключевые таблицы
	вебхук	
matching	потребности работодателя, разбор свободного текста, рекомендованные категории, подборка со счётом и объяснением	needs
invitations	приглашения, машина статусов, квота, рыночная подсказка, раскрытие контактов, письма	invitations, invitation_events
vacancies	вакансии с вилкой, публикация, лента с фильтрами, отклики со статусами, контакты после отклика, письма	vacancies, applications
tasks	регулярные короткие задания от работодателя, раздача, ответы, оценка 0–3, свежесть профиля, периодические работы (python -m app.tasks)	regular_tasks, task_submissions
admin	дашборд, пользователи, компании (скоринг надёжности), справочники и настройки, банк задач (метрики, утечки, калибровка, одобрение черновиков), тесты из вакансий, жалобы, журнал	audit_log, reports, vacancy_tests
eval	графики валидации и проверка подбора по CSV пар «потребность — кандидат»	—
nlp	разбор PDF-резюме, текста потребности и вакансии правилами и словарями; LLM только улучшает результат	—

Схема данных целиком — docs/architecture/er.md (ER-диаграмма и решения по хранению). Миграции — Alembic, backend/alembic/versions/; бэкенд применяет их сам при старте (alembic upgrade head).

1.5. Фронтенд

React 18 + TypeScript + Vite, React Router 7, TanStack Query 5, Tailwind CSS 4, Recharts. Одно приложение на три роли: маршруты /candidate/*, /recruiter/*, /admin/* под проверкой роли. Типы API генерируются из OpenAPI (npm run gen:api → src/shared/api/schema.ts), поэтому контракт фронта и бэкенда один.

Режим VITE_USE_MOCKS: true — прототип работает без бэкенда на моках MSW (данные из тех же фикстур data/seed, что и сид бэкенда); false — запросы идут в настоящий API. Access-токен хранится только в памяти вкладки, refresh — в httpOnly cookie; при 401 клиент тихо обновляет токен. Цвета и шрифты — по брендбуку ФСП (#402FFF, JetBrains Mono, Inter), только через токены src/index.css. Вёрстка адаптивная (мобильная версия).

1.6. Карта экранов

Публичные: / лендинг с двумя входами («Ищу работу» / «Ищу специалистов»), /login (почта + пароль, «Войти через ФСП ID»), /register (роль, почта, пароль, согласие), /verify-email, /reset-password, /auth/fsp-callback.

Соискатель /candidate/…	Рекрутер /recruiter/…	Администратор /admin/…
(главная): категория, новые приглашения, задание, полнота профиля | (главная): потребности, приглашения по статусам, квота, отклики	`(дашборд): пользователи, тесты, приглашения, конверсия, флаги утечек | |onboarding: профиль → опрос → грейд → тест → результат |company: профиль компании, рекрутеры |users: роль, блокировка, журнал | | test/:sessionId,test/:sessionId/result| needs,needs/new: потребность формой или текстом |companies: скоринг надёжности, блокировка | |profile: форма, загрузка PDF, «Скачать PDF-профиль» |needs/:id: категории → подборка с «почему» и полосками счёта, фильтры |catalog: справочники, пороги θ, кулдауны, квота, веса | | grade: статус, история, тест ниже/выше с кулдауном | candidates,candidates/:id: банк и обезличенный профиль |items,items/:id: банк задач, 3 варианта, вывод /	

Соискатель /candidate/…	Рекрутер /recruiter/…	Администратор /admin/…
	одобрение | |invitations,invitations/:id: принять / отклонить с причиной, контакты | модальное окно приглашения: вилка, способ связи, задание, рыночная подсказка, квота | vacancy-tests: текст вакансии → темы → тест | |vacancies,applications: лента и отклики |invitations: статусы, причины отказов |validation: графики валидации | |tasks: регулярное задание | vacancies,vacancies/:id: вакансии и отклики |reports: жалобы | |fsp: достижения или «нет данных ФСП» + привязка |tasks: короткие задания, оценка ответов |audit: журнал действий | |settings`: приватность, согласия, ФСП ID, пауза поиска	

1.7. Основные потоки

Сквозной сценарий: от регистрации до контакта

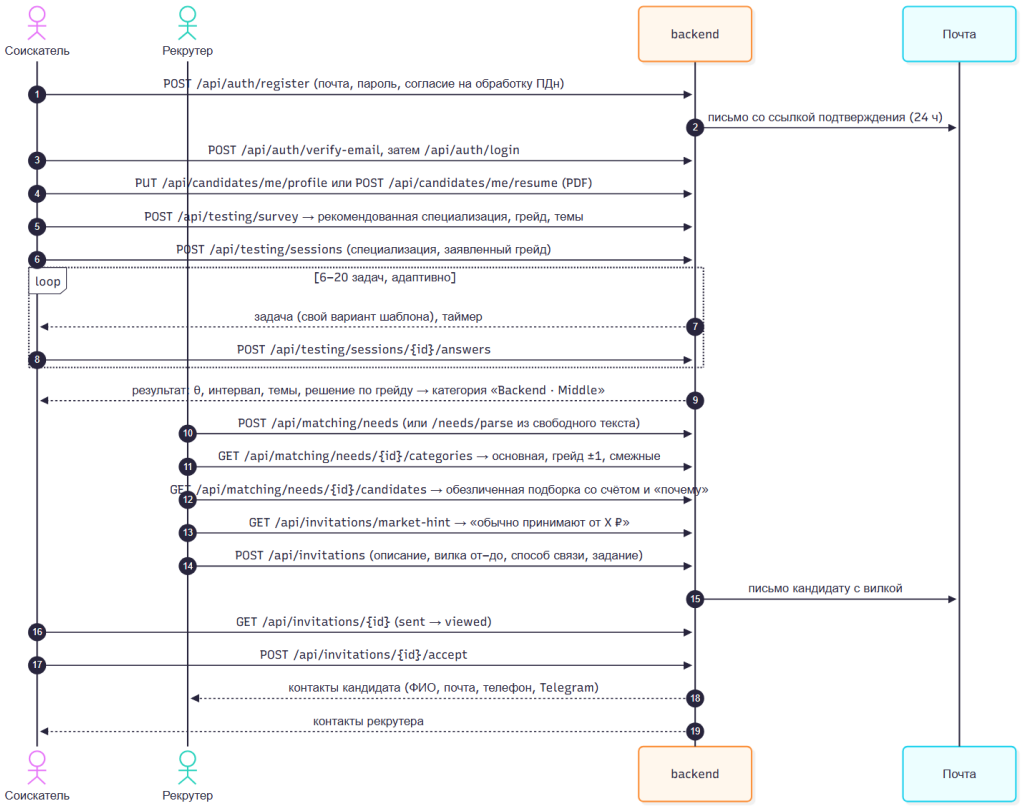


Диаграмма 3

Дополнительный сценарий — отклик на вакансию: кандидат без приглашений находит вакансию в ленте (`GET /api/vacancies`) и откликается (`POST /api/vacancies/{id}/applications`); контакты рекрутеру открываются сразу, потому что кандидат проявил инициативу сам.

Кандидат без истории ФСП

Обрабатывается как обычный путь: опрос, тест и категория не зависят от ФСП. В выдаче у такого кандидата фактор «Достижения ФСП» равен 0 — бонуса нет, штрафа нет, остальные факторы не меняются. Работодатель видит нейтральное «Нет данных ФСП». Подробнее — раздел 5.

1.8. Безопасность и 152-ФЗ

- **Согласия.** При регистрации обязательно согласие на обработку ПДн; публикация профиля работодателям — отдельное согласие. Без него кандидата нет ни в банке, ни в подборках; отзыв действует сразу. Версия текста согласия хранится (`platform_settings.consent_version`).
- **Обезличивание.** До принятия приглашения рекрутер видит псевдоним «Кандидат #XXXX», категорию, темы теста, стек, бейджи ФСП — без ФИО, почты, телефона, мессенджеров, минимальной зарплаты. Город, стаж и ссылки — только если кандидат разрешил в настройках приватности; блок ФСП можно скрыть. Контакты — отдельный объект, он заполняется только после принятия приглашения или отклика самого кандидата.
- **Журнал.** Раскрытие контактов, смены грейда, блокировки, вывод шаблонов и изменения настроек пишутся в `audit_log`.
- **Пароли и токены.** Пароли — scrypt с солью на каждый пароль (стандартная библиотека Python). Access-токен JWT (HS256) живёт 15 минут; refresh-токен — в httpOnly cookie на пути `/api/auth`, в БД хранится только его хеш, при каждом обновлении он меняется (ротация). Не больше 5 неудачных входов за 15 минут на пару «почта + IP».
- **ФСП ID.** Токен Keycloak проверяется на бэкенде: подпись по JWKS, `iss`, `aud`, срок, `nonce`; `state` и PKCE (S256) — в подписанной httpOnly cookie. Вебхук реестра принимается только с верной HMAC-подписью и не старше 5 минут.
- **Демо-заголовки** `X-Demo-Role` / `X-Demo-User-Id` работают только при `DEMO_AUTH_ENABLED=true` (dev); на стенде выключены.
- **Ключ ответа не покидает сервер:** клиент получает только формулировку варианта, ключ, id шаблона и seed остаются в БД (подробнее — раздел 2).

1.9. Масштабирование и интеграция

- Бэкенд без состояния в памяти (сессия теста целиком в БД, JWT), поэтому его можно запускать в нескольких экземплярах за прокси.
- Справочники и веса ранжирования — данные в БД, админ меняет их без релиза.
- Внешние системы подключаются через HTTP: реестр ФСП (клиент + вебхук), Keycloak (OIDC), SMTP, LLM; ATS работодателей — API-токен компании, выгрузка и исходящие вебхуки с HMAC-подписью и повторами (раздел 3.13, `docs/concepts/antifraud_ats.md`).
- Периодические работы (раздача заданий, снижение свежести, повтор вебхуков) — отдельные команды `python -m app.tasks …` и `python -m app.integrations` для cron (раздел 8.6); бэкенд при этом остаётся без фоновых потоков.
- API полностью описан в OpenAPI (раздел 6) — по нему можно сгенерировать клиента.
- Производительность: целевой ориентир Т3 — десятки одновременных пользователей. Стенд работает на сервере 2 vCPU / 4 ГБ. Отдельный нагрузочный замер мы не проводили; при росте нагрузки бэкенд можно запустить в нескольких экземплярах (см. выше).

2. Механика тестирования и устойчивость к распространению заданий

Подробное техническое описание с формулами и полной симуляцией — `docs/testing/engine.md`; состав банка по специализациям — `docs/testing/bank/`; правила опроса — `docs/testing/survey.md`. Код движка: `backend/app/testing/engine/` (чистая библиотека Python + numpy), подключение к API и БД: `backend/app/testing/`.

2.1. Проблема и наш ответ

ТЗ называет две ловушки:

1. **Единый набор заданий** быстро расходится по рынку — ответы выучивают.
2. **Уникальные задания от нейросети** под каждого кандидата не гарантируют одинаковую сложность — результаты разных людей нельзя честно сравнить, а правильность ключа никто не проверял.

Наш подход — **параметрические шаблоны с измеренной сложностью + адаптивный тест на модели IRT + автоматический детектор утечек**. Вариативность есть у каждого кандидата, но она управляемая: генератор меняет данные внутри шаблона, а сложность шаблона измерена по ответам и проверяется.

2.2. Путь кандидата

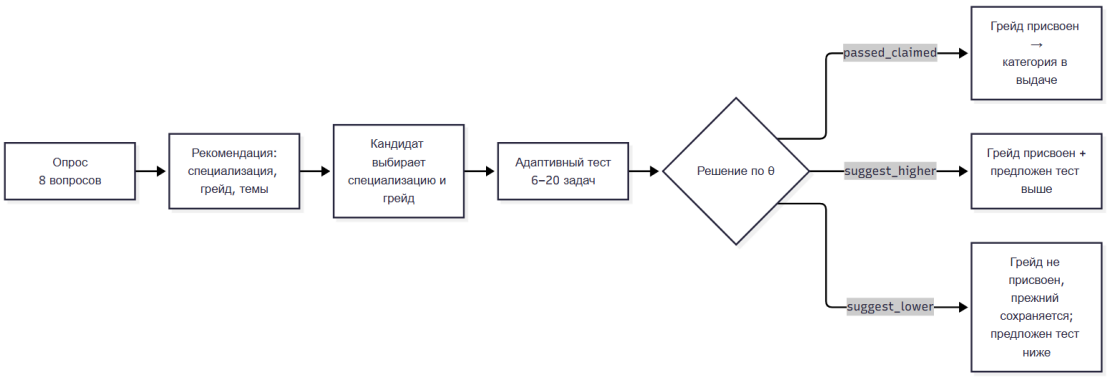


Диаграмма 4

1. **Опрос** (`GET/POST /api/testing/survey`) — подсказка, а не оценка. Голоса вариантов ответа с весами вопросов дают рекомендуемую специализацию (и две альтернативы с долей), опыт — стартовый грейд, ответы и ключевые слова свободного ответа — до пяти тем теста.
2. **Выбор** — кандидат может согласиться или выбрать другую специализацию и грейд.
3. **Тест** (`POST /api/testing/sessions`, далее `GET …/{id}` и `POST …/{id}/answers`) — адаптивный, задачи подбираются под текущую оценку уровня.
4. **Итог** (`GET …/{id}/result`) — θ, 90% доверительный интервал, разбор по темам («сильная» / «слабая»), решение по грейду. Категория появляется в профиле и в выдаче работодателю.

2.3. Банк параметрических шаблонов

Задача — это **шаблон**, а не текст. Шаблон хранит специализацию, тему, целевой грейд, формат, генератор, проверку, параметры IRT `a` и `b`, статус и лимит времени. `generate(seed)` детерминирован: один seed — один вариант, его можно восстановить и перепроверить. Ответ вычисляет сам генератор, код кандидата не запускается.

Банк MVP: 192 шаблона — по 32 на каждую из 6 специализаций (30 основных + 2 примера), 52 темы. Каждый шаблон даёт **не меньше 50 разных вариантов** (backend и frontend — от 100); это проверяет общий тест банка `test_engine_bank_full.py`.

Формат	Шаблонов	Пример (что меняется от варианта к варианту)
`numeric` — число	49	сколько строк вернёт JOIN (таблицы, тип JOIN)
`single_choice` — один ответ	40	сложность алгоритма (код циклов, имена, порядок вариантов)

Формат	Шаблонов	Пример (что меняется от варианта к варианту)
`code_output` — что выведет код	40	порядок вывода event loop JS (набор `console.log` / `setTimeout` / `Promise`)
`multi_choice` — несколько ответов	24	граничные значения в QA (поле, диапазон, метод)
`find_bug_line` — строка с ошибкой	14	ошибка в Kotlin или в Dockerfile (вид ошибки, имена, лишние строки)
`ordering` — порядок шагов	13	жизненный цикл Android Activity (сценарий, перемешивание)
`short_text` — короткий ответ	12	адрес сети / broadcast по CIDR (IP, маска)

Грейды шаблонов: Стажёр 37, Junior 53, Middle 59, Senior 43.

В админке («Банк задач») видно 222 записи: эти 192 шаблона движка и 30 демо-записей из фикстур (`data/seed/items.json`) — они показывают экраны метрик и предпросмотра, но генератора в движке у них нет, в адаптивный тест и валидацию они не попадают.

Ключи сверены с настоящими системами, а не «на глаз»: SQL-шаблоны выполняются в SQLite (и в PostgreSQL 16 для шаблонов про транзакции), JS — в Node.js, Python-код — настоящим интерпретатором, CIDR — модулем `ipaddress`, CSS — независимым подсчётом специфичности. Отдельный тест следит, чтобы позиция верного ответа не повторялась (ни один ключ не встречается больше чем в половине вариантов). Подробно по каждому шаблону — `docs/testing/bank/<spec>.md`.

Пример: два кандидата получают один шаблон «граничные значения» и видят разные задачи — «Число гостей от 1 до 17» и «Срок кредита от 4 до 114 мес.» — с разными числами и порядком вариантов. Ответ одного другому не подходит.

2.4. Модель IRT и оценка уровня

Двухпараметрическая логистическая модель (2PL):

```
P(верно | θ) = 1 / (1 + e^(-a·(θ - b)))
```

- `θ` — уровень кандидата (по популяции среднее 0, разброс 1);
- `b` — сложность (при `θ = b` шанс 50%), `a` — различающая сила.

Уровень оценивается методом EAP на сетке `θ` от −4 до 4 (81 точка); стандартная ошибка SE — разброс апостериорного распределения. Априор — нормальный с центром по заявленному грейду (Стажёр −1,5; Junior −0,5; Middle 0,5; Senior 1,5): старт быстрее, но после 6 ответов данные перевешивают априор. Стартовые `a`, `b` шаблона задаёт эксперт по грейду, затем их уточняет калибровка (2.8).

2.5. Адаптивный тест (CAT)

Каждый шаг:

1. текущая `θ` — по проверочным ответам с априором заявленного грейда;
2. из банка убираются уже показанные задачи, задачи из прошлых попыток кандидата, темы, которых в тесте уже 6 (баланс тем), и — при большом банке — задачи, показанные больше чем в 25% сессий (контроль экспозиции);
3. для каждой задачи — информация Фишера $a^2 \cdot P \cdot (1 - P)$ в текущей `θ` с понижением за повтор темы (деление на `1 + 0,1·k`);
4. **выбирается случайная задача из пяти самых информативных** (randomesque) — даже при одинаковых ответах два кандидата получают разные наборы задач.

Параметр	В продукте (API)	Где задаётся
Остановка по точности	SE < 0,30	`TESTING_CAT_SE_STOP`
Максимум задач	20	`TESTING_CAT_MAX_ITEMS`
Минимум задач	6	`TESTING_CAT_MIN_ITEMS`
Лимит времени на задачу	180 с (у шаблона может быть свой)	шаблон
Контроль экспозиции	доля показов ≤ 25%, включается при ≥ 100 активных шаблонах в специализации	`TESTING_EXPOSURE_MIN_BANK`, `exposure_max_share` в настройках
Сессия брошена	нет ответа дольше лимита + 30 мин → неудачная попытка	`TESTING_ABANDON_AFTER_MIN`

Непроверочные (пилотные) задачи вставляются в 1–2 случайные позиции среди первых шести показов. Кандидат их не отличает, на θ они не влияют, ответы идут в калибровку — так новые шаблоны (в том числе созданные по вакансиям) набирают статистику без риска для оценки.

Время считает сервер по своим часам от выдачи задачи; ответ после лимита засчитывается как неверный. Время каждого ответа сохраняется — оно нужно детектору утечек.

Ключ не покидает сервер. Клиент получает только формулировку, варианты и код. Ключ, id шаблона и seed остаются в БД (`test_answers`), наружу уходит непрозрачный `instance_id`. Верно/неверно во время теста не сообщается, правильные ответы после теста не показываются.

Состояние сессии целиком сериализуется в `test_sessions.engine_state` (JSON), случайность выводится из seed сессии и номера шага: сессию можно прервать, восстановить и получить тот же результат; бэкенд не держит состояния в памяти.

2.6. Грейд и правила частоты

Пороги θ (настраиваются админом по каждой специализации, `platform_settings.grade_thresholds`): Junior от −1,0; Middle от 0,0; Senior от 1,0; ниже −1,0 — Стажёр. При нормальном распределении уровней это примерно 16% / 34% / 34% / 16%.

Исход	Условие	Что происходит
`passed_claimed`	θ не ниже порога заявленного грейда	грейд присваивается
`suggest_higher`	нижняя граница 90% ДИ (θ − 1,645·SE) выше порога следующего грейда	грейд присваивается, предлагается тест на грейд выше
`suggest_lower`	θ ниже порога заявленного грейда	грейд не присваивается (прежний сохраняется), предлагается тест на грейд ниже

Принудительного понижения нет — как требует ТЗ. Тест на Стажёра пройти можно всегда.

Частота (настройки `grade_change_cooldown_days`, `retry_cooldown_days`):

- **смена грейда — не чаще раза в 90 дней** («раз в три месяца» из ТЗ);
- повтор неудачной попытки на тот же грейд — через 30 дней;
- первое присвоение грейда сменой не считается: после первого теста можно сразу пройти тест ниже (если не прошёл) или выше (если предложено);
- подтверждение текущего грейда сменой не считается.

Запрет отвечает `409 cooldown_active` с причиной (`details.reason_code`) и датой, когда станет можно (`details.available_at`); экран «Категория и грейд» показывает это заранее.

2.7. Детектор утечек

Работает по журналу ответов, по каждому шаблону, в окне последних 300 ответов.

1. **IRT-сигнал.** Ожидаемая доля верных — среднее P(верно) по θ отвечавших, причём θ считается **без этой задачи** (`test_answers.theta_loo`), чтобы слитая задача сама не подтягивала оценку. Флаг: не меньше 30 ответов, $z \geq 3{,}5$ и рост доли верных не меньше 10 п. п.
2. **Сигнал скорости.** «Быстрый верный» — верный ответ быстрее 35% медианного времени по задаче. Флаг: не меньше 4 таких ответов и вероятность случайно получить столько (хвост Пуассона) меньше 10^{-4}.

Помеченный шаблон выводится из оборота (`retired`), событие пишется в журнал, админ видит флаг в банке задач. Запуск — автоматически после каждой 50-й завершённой сессии (`TESTING_LEAK_SCAN_EVERY`) и вручную: `POST /api/admin/items/leak-scan`.

Аномалии кандидата: два и больше быстрых верных ответа на задачи, сложные для его уровня (шанс ниже 60% по θ без этой задачи), плюс индекс согласия ответов $l_z < -3$. Это повод для ручной проверки, а не автоматическое наказание.

2.8. Калибровка и жизненный цикл шаблона



Диаграмма 5

Пересчёт `a`, `b` (`POST /api/admin/items/recalibrate`) — логистическая регрессия ответа на θ отвечавших (без этой задачи), метод Ньютона со слабым априором вокруг текущих значений, рамки $a \in [0{,}2;\ 4]$, $b \in [-4;\ 4]$. Пересчитываются шаблоны с не меньше чем 30 ответами

(`TESTING_CALIBRATION_MIN_RESPONSES`). Пилот становится рабочим, когда: не меньше 150 ответов, оценка сошлась, a ≥ 0,4, |b| ≤ 3,5, SE(b) ≤ 0,35.

2.9. Задания и тесты из текста вакансии

ТЗ просит показать, как система формирует задания по описаниям вакансий. Админ вставляет текст вакансии (`POST /api/admin/vacancy-tests`, экран `/admin/vacancy-tests`):

1. правила определяют специализацию и грейд (с объяснением «по каким словам»);
2. словарь тем находит темы с уверенностью;
3. по каждой теме из банка берётся шаблон ближайшего грейда — получается тест;
4. если темы в банке нет — создаётся **черновик** шаблона (с ключом LLM его пишет модель, ответ проверяется Pydantic; без ключа — заготовка словаря). Черновик одобряет админ, дальше он идёт в тесты как непроверочный и калибруется.

Проверка на 5 образцах вакансий (`data/samples/vacancies/`): специализация верна 5 из 5, грейд 5 из 5, ожидаемые темы найдены 25 из 25 — `docs/validation/vacancy_tests.md`.

2.10. Почему механика устойчива к распространению заданий

Защита	Что делает	Чем подтверждено
Параметризация	у каждого кандидата свой вариант; слитый ответ подходит к чужому варианту лишь случайно	утечка 20% банка, текущий банк 192 шаблона: завышение грейда у «читеров» 39,0% → 12,7% (фон честных — 10–11%); в симуляции движка 85% → 14%
Адаптивность + случайный выбор из топ-5	разные кандидаты видят разные наборы задач; «полного списка вопросов» не существует	у каждой сессии свой набор
Контроль экспозиции	ни одна задача не показывается больше чем в 25% сессий (при банке от 100 шаблонов в направлении)	максимум показов в симуляции движка — 25,0%
Детектор утечек	слитая задача выводится из оборота автоматически	текущий банк: найдены все 36 слитых шаблонов, 2 ложных из 156 (точность 95%), завышение 39% → 13%; симуляция движка: полнота 94%, точность 100%
Непроверочные задачи	банк постоянно пополняется, новые шаблоны не влияют на оценку до калибровки	тест «пилотные задачи не влияют на θ»
Ключ не покидает сервер	клиенту не уходят ответ, id шаблона и seed; ответы после теста не показываются	тест публичной части варианта
Время по часам сервера	быстрые верные ответы видны детектору и в аномалиях кандидата	детектор кандидатов без параметризации: точность 99%, полнота 71% (текущий банк); в симуляции движка — 83% без ложных обвинений

Два уровня страхуют друг друга: с параметризацией детектору почти нечего ловить (слитый ответ редко подходит) — и вреда тоже нет; если какой-то шаблон всё же один на всех, его находит детектор.

2.11. Почему не «одна нейросеть генерирует задания каждому»

У сгенерированной на лету задачи нет измеренных `a` и `b`: два кандидата получают задания неизвестной и разной трудности, их баллы несопоставимы; за правильность ключа никто не ручается. У нас нейросеть используется там, где она безопасна: по тексту вакансии она предлагает **черновики** шаблонов. Черновик проверяет человек, затем шаблон проходит пилот и калибровку — и только после этого влияет на оценки. Без ключа LLM механика работает полностью.

2.12. Результаты проверки механики

Платформа на текущем банке — 1200 синтетических кандидатов с известным истинным θ проходят настоящий онбординг (тест на заявленный грейд, ниже или выше по правилам 2.6) на рабочем банке 192 шаблона с настройками стенда (SE < 0,30, до 20 задач), seed 20261011; воспроизводится `make validate`, подробно — `docs/validation/report.md`:

Метрика	Значение
Корреляция θ с истинным θ / RMSE	0,940 / 0,358

Метрика	Значение
Грейд по θ одного теста: точно / ±1	80,2% / 99,8%
Итоговая категория: точно / ±1	**78,8% / 100,0%** (по заявленному грейду было бы 64,9%)
Категория точно, если истинный θ дальше 0,25 от порога	84,5%
Средняя длина теста / средняя итоговая SE	20,0 задачи / 0,374
Повтор онбординга: та же категория / каппа Коэна	75,6% / 0,663
Различающая сила: средняя r_pb по 192 шаблонам / доля > 0,2	0,430 / 100%
Утечка 20% банка: завышение у читеров без защиты → с параметризацией	39,0% → 12,7% (фон 10,7%)
Детектор утечек (шаблон один на всех): полнота / точность	100% / 95%

Движок на большом синтетическом банке — 1000 кандидатов, банк 240 задач, правило по умолчанию движка (SE < 0,35 или 15 задач), seed 20261011; воспроизводится командой `PYTHONPATH=backend python -m app.testing.engine.simulate --seed 20261011`:

Метрика	Значение
Корреляция оценки θ с истинным θ	0,945
RMSE θ	0,332
Грейд точно / ±1	78,4% / 100%
Грейд точно, если истинный θ дальше 0,25 от порога	88,5%
Средняя длина теста (стоп SE < 0,35 или 15 задач)	12,8
Повтор: тот же грейд / каппа Коэна	67,9% / 0,558
Повтор: то же решение по заявленному грейду / каппа	86,4% / 0,587
Повтор при правиле продукта (SE < 0,30 или 20 задач): тот же грейд / каппа	73,6% / 0,636
Различающая сила: средняя a банка; r_pb на линейной форме 30 задач	1,32; 0,348 (100% задач > 0,2)
Ошибка калибровки ±0,2 → корреляция	0,932
Калибровка 30 пилотов × 300 ответов: RMSE a / b; переведено в рабочие	0,198 / 0,228; 90%

Почему цифры двух таблиц различаются: на большом банке тест останавливается раньше (12,8 задачи) и выбирает самые информативные задачи; на текущем банке из 32 шаблонов в направлении цель SE < 0,30 недостижима, поэтому все тесты идут до 20 задач. Повтор в первой таблице — совпадение итоговой категории после всего онбординга, во второй — грейда по одному тесту. Подробно, с графиками и разбором подбора — раздел 4.

2.13. Ограничения и что дальше

- **Банк MVP — 32 шаблона на специализацию.** На нём тест всегда идёт 20 задач и заканчивает со средней SE 0,37 (цель 0,30 не достигается), показывает больше половины банка, повтор даёт ту же категорию в 76% случаев; контроль экспозиции поэтому выключен (включается при ≥ 100 активных шаблонах). Точность ещё растёт с размером банка (16 шаблонов — грейд точно 75,0%, 32 — 79,8%). По симуляции движка 120+ шаблонов на специализацию дают SE 0,34–0,37 за 12–15 задач — это главный путь к стабильности. Параметризация защищает и маленький банк.
- **Равенство вариантов внутри шаблона** по сложности — допущение; проверяется на реальных ответах калибровкой по группам вариантов, шаблоны с разбросом стоит дробить.
- **Модель времени** в симуляции — допущение; на реальных данных пороги сигнала скорости нужно проверить по истории (код уже оценивает фон по всему банку).
- **Пороги грейдов** −1 / 0 / 1 — стартовые; после набора данных их стоит согласовать с экспертной оценкой.

3. Механика подбора и логика категоризации

Код: `backend/app/matching/` (потребности, категории, подборка, счёт — `scoring.py`), `backend/app/invitations/` (приглашения, квота, рыночная подсказка, контакты), `backend/app/candidates/` (обезличенная карточка, банк кандидатов), `backend/app/fsp/` (ФСП-скор), `backend/app/nlp/` (разбор текста потребности, сравнение стека), `backend/app/vacancies/` (вакансии и отклики), `backend/app/tasks/` (регулярные задания и свежесть профиля), `backend/app/companies/` и `backend/app/integrations/` (антифрод, ATS).

3.1. Идея: работодатель приходит сам

Классическая площадка: кандидат рассылает отклики, работодатель разбирает сотни резюме. У нас наоборот — по модели LHH из ТЗ:

1. кандидат один раз проходит опрос и тест → получает **категорию**;
2. работодатель описывает потребность → видит **рекомендованные категории** и внутри них **ранжированную подборку с объяснением**;
3. работодатель выбирает конкретного человека и отправляет **приглашение с зарплатной вилкой** — кандидат видит условия до начала общения;
4. кандидат принимает или отклоняет с причиной; **контакты открываются только после принятия**.

Приглашение — ценный ресурс: у рекрутера квота активных приглашений, вилку ниже ожиданий кандидата отправить нельзя, отказ возвращается с причиной. Так нет спама и есть обратная связь обеим сторонам.



Диаграмма 6

3.2. Категоризация кандидата

Категория = специализация × грейд, код `"<spec>:<grade>"` (например `backend:middle`, подпись «Backend · Middle»).

Справочник	Значения
Специализации (6)	`backend` Backend, `frontend` Frontend, `mobile` Mobile, `data_ml` Data Science / ML, `qa` QA, `devops` DevOps
Грейды (4)	`intern` Стажёр, `junior` Junior, `middle` Middle, `senior` Senior
Категории	24 = 6 × 4
Стек	справочник технологий с привязкой к специализациям и синонимами (админ пополняет)

Откуда берётся категория:

- **специализация** — выбирает кандидат перед тестом (опрос рекомендует, но не навязывает); тест идёт по банку этой специализации;
- **грейд** — только по результату адаптивного теста (θ против порогов), см. раздел 2.6. Самоописание в резюме на грейд не влияет; принудительного понижения нет;
- кандидат попадает в выдачу, только если у него есть грейд, дано согласие на публикацию, поиск не на паузе и учётная запись не заблокирована.

Резюме (форма или PDF с автораспознаванием) даёт стек, опыт, роли, контакты — это **дополнительные** данные для ранжирования внутри категории и для PDF-профиля, но не основание для категории. Так выполняется требование ТЗ: «выдача формируется на основании категорий, присвоенных по итогам опроса и тестирования, а не самоописанных данных резюме».

3.3. Потребность работодателя

Рекрутер описывает потребность формой (`POST /api/matching/needs`) или свободным текстом: `POST /api/matching/needs/parse` разбирает фразу вида «нужен сеньор на Go в финтех, удалёнка, 300–400к» в поля, рекрутер проверяет и сохраняет. Разбор — правилами и справочником синонимов; с

ключом LLM — точнее. Проверка на 12 формулировках с эталоном — 12 из 12 по всем полям (`docs/validation/need_parsing.md`).

Поля: название, специализация, грейд, обязательный и желательный стек (коды справочника, неизвестные — `422 unknown_skills`), описание команды, формат работы, город, **вилка от–до (обязательно, от ≤ до)**.

3.4. Рекомендованные категории

`GET /api/matching/needs/{id}/categories` — основная категория потребности, соседние грейды (±1) и смежные специализации того же грейда, у каждой — число кандидатов в выдаче:

Специализация потребности	Смежные
Backend	DevOps, Data Science / ML
Frontend	Mobile, Backend
Mobile	Frontend
Data Science / ML	Backend
QA	Backend, Frontend
DevOps	Backend

Если в основной категории мало людей, рекрутер видит, куда расширить поиск, и переключает категорию одним кликом.

3.5. Ранжирование внутри категории

`GET /api/matching/needs/{id}/candidates?category=…` считает для каждого кандидата категории **счёт 0..1** — взвешенную сумму шести факторов (`backend/app/matching/scoring.py`):

```
счёт = Σ value(фактор) × weight(фактор),   value ∈ [0; 1],   Σ weight = 1
```

Фактор	Вес по умолчанию	Как считается value
Стек (`stack`)	0,30	покрытие стека потребности: обязательные технологии — 75%, желательные — 25%; точное совпадение или синоним — 1, смежная технология — частичный вклад из справочника смежности (например, FastAPI ↔ Django = 0,5)
Результат теста (`theta`)	0,25	Φ(θ) — доля специалистов с уровнем ниже (θ ~ N(0, 1)); без теста — 0
Достижения ФСП (`fsp`)	0,15	ФСП-скор / 100, если кандидат показывает достижения; нет истории — 0 (бонуса нет, штрафа нет)
Свежесть профиля (`freshness`)	0,10	0..1, растёт за ответы на регулярные задания и их оценку, со временем падает вдвое за 30 дней (3.12)
Зарплата (`salary`)	0,10	1 — минимум кандидата не выше «от» вилки; 0,7 — не выше «до»; иначе 0
Формат работы (`work_format`)	0,10	1 — совпадает; 0,5 — кандидат на гибриде, а нужна удалёнка или гибрид; иначе 0

Веса хранятся в настройках платформы (`platform_settings.ranking_weights`), админ меняет их на экране «Справочники» (`PUT /api/admin/settings`) без релиза; при расчёте они нормируются к сумме 1, поэтому сумма вкладов точно равна счёту. При равном счёте порядок стабильный (по id кандидата).

Почему такие веса: категория уже гарантирует нужную специализацию и подтверждённый грейд, поэтому внутри неё главное — **совпадение стека** и **сила результата теста** (вместе 55%); **ФСП** — объективное внешнее подтверждение, как требует ТЗ («выше те, у кого есть подтверждённые достижения ФСП»), но не настолько тяжёлое, чтобы кандидат без истории ФСП не мог оказаться в топе; зарплата, формат и свежесть — практические условия выхода на контакт.

3.6. Объяснимость: «почему в подборке»

Каждая строка подборки содержит:

- **разбивку счёта** (`breakdown`): по каждому фактору value, вес и вклад — на экране это полоски;
- **2–4 причины текстом** (`reasons`) по самым весомым факторам, например: «Стек: 3 из 4 обязательных (Go, PostgreSQL, Kafka), плюс Kubernetes», «Тест сильнее, чем у 87 % специалистов; сильные темы: Базы данных и SQL, Конкурентность», «Подтверждено ФСП: призёр всероссийского хакатона по продуктовому программированию, 2026», «Ожидания по зарплате укладываются в вилку», «Готов работать: удалённо»;

- `salary_fit` — укладывается ли минимум кандидата в вилку (без раскрытия суммы);
- статус последнего приглашения этой компании кандидату, чтобы не пригласить дважды.

Пример реального ответа (демо-данные, категория «Backend · Senior»):


```
{
  "score": 0.4017,
  "reasons": [
    "Тест сильнее, чем у 95 % специалистов; сильные темы: HTTP и REST API, Конкурентность и
асинхронность",
    "Категория по тесту: Backend · Senior"
  ],
  "breakdown": [
    {"factor": "stack", "label": "Стек", "value": 0.2625, "weight": 0.3, "contribution":
0.0788},
    {"factor": "theta", "label": "Результат теста", "value": 0.9474, "weight": 0.25,
"contribution": 0.2369},
    {"factor": "fsp", "label": "Достижения ФСП", "value": 0.0, "weight": 0.15, "contribution":
0.0},
    {"factor": "freshness", "label": "Свежесть профиля", "value": 0.36, "weight": 0.1,
"contribution": 0.036},
    {"factor": "salary", "label": "Зарплата", "value": 0.0, "weight": 0.1, "contribution":
0.0},
    {"factor": "work_format", "label": "Формат работы", "value": 0.5, "weight": 0.1,
"contribution": 0.05}
  ],
  "salary_fit": false
}
```


3.7. Уточнение запроса без потери подборки

Фильтры подборки — стек (все выбранные технологии), «есть ФСП», формат работы, город — применяются поверх уже посчитанного списка: ответ содержит `total_before_filters` и `total`, порядок не пересчитывается, сброс фильтров возвращает исходную подборку. Отдельно есть **банк кандидатов** (`GET /api/candidates`) — поиск по всей базе с фильтрами по специализации, грейду, категории, стеку, наличию ФСП, формату и городу, сортировка по тесту, ФСП, свежести или новизне.

3.8. Обезличенный профиль

До принятия приглашения рекрутер видит (`GET /api/candidates/{id}`, карточки подборки):

Видно	Скрыто
псевдоним «Кандидат #XXXX», категория, θ и темы теста («сильные» / «слабые»), стек, роли в команде, форматы работы, бейджи и ФСП-скор с пометкой «подтверждено ФСП»	ФИО, почта, телефон, Telegram, минимальная зарплата, ФСП ID; город, стаж и ссылки — если кандидат не разрешил их показывать; блок ФСП — если кандидат его скрыл

Контакты — отдельный объект `contacts`; он заполняется только после принятия приглашения этой компании или если кандидат сам откликнулся на её вакансию.

3.9. Приглашение: выход на контакт

`POST /api/invitations` — без обязательной привязки к вакансии (можно указать потребность или вакансию для контекста):

Поле	Обязательно	Правило
описание предложения	да	
вилка `salary_from`–`salary_to`, ₽	да	от > 0, от ≤ до (`422 validation_error`, `salary_range_invalid` в `details.errors`)
способ связи	да	`email`, `phone`, `telegram`, `other` + значение
компания	—	подставляется из профиля рекрутера
короткое задание	нет	текст задания для кандидата

Проверки при отправке:

Ситуация	Ответ
максимум вилки ниже минимальной зарплаты кандидата	`422 salary_below_expectations` — **сумма ожиданий не**

Ситуация	Ответ
	раскрывается, только факт
у кандидата уже есть активное приглашение от этой компании	`409 already_invited`
квота рекрутера исчерпана (по умолчанию 20 активных, т.е. «отправлено» и «просмотрено»)	`409 quota_exceeded` с подсказкой отозвать неактуальные
кандидат скрыл профиль, на паузе или не найден	`404 not_found`

Кандидату уходит письмо с компанией и вилкой («Ваши контакты компания увидит, только если вы примете приглашение»). Приглашение действует 7 дней (`invitation_ttl_days`).

Статусы

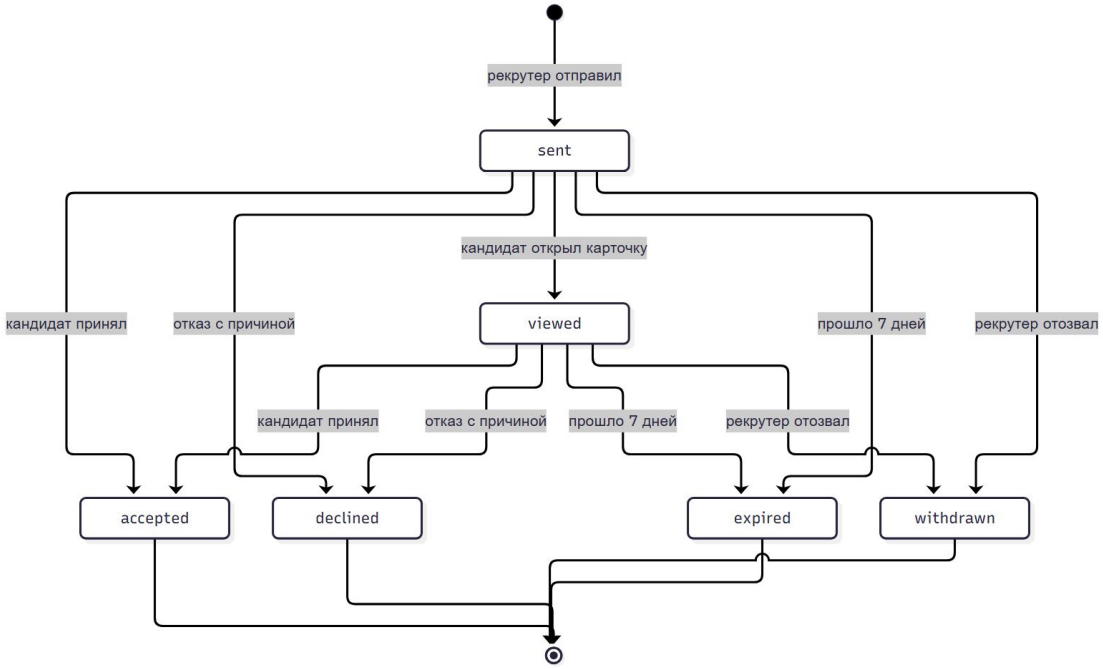


Диаграмма 7

Недопустимый переход — `409 invalid_transition`. Каждое изменение пишется в журнал событий приглашения (`invitation_events`); обе стороны видят актуальный статус и историю (`GET /api/invitations/{id}`). Истечение срока применяется при любом обращении к приглашениям.

Отказ с причиной (`POST /api/invitations/{id}/decline`): «Мала зарплата», «Не тот стек», «Не интересна сфера», «Не ищу работу сейчас», «Другое» (для «Другое» комментарий обязателен). Рекрутер получает письмо с причиной и видит её в списке — это обратная связь о рынке.

Раскрытие контактов (`POST /api/invitations/{id}/accept`): рекрутер видит ФИО, почту, телефон и Telegram кандидата, кандидат — контакты рекрутера; событие `contacts_revealed` пишется в журнал приглашения и в `audit_log`, рекрутеру уходит письмо с контактами.

3.10. Рыночная подсказка

`GET /api/invitations/market-hint?category=…&salary_from=…&salary_to=…` — показывается в окне приглашения:

- если в категории принято не меньше 3 приглашений — медиана принятых вилок («В категории Backend · Middle обычно принимают от 250 000 ₽»);
- иначе — оценка по минимальным ожиданиям кандидатов категории (медиана, округление до 10 000 ₽);
- **вероятность принятия** с введённой вилкой: доля кандидатов категории, чей минимум не выше верхней границы, с поправкой на то, насколько «от» ниже медианы; ограничена 5–95%. При малой выборке подсказка честно говорит «пока мало данных».

Цель — чтобы рекрутер сразу ставил рыночную вилку, а не тратил приглашения из квоты.

3.11. Отклик на вакансию — дополнительный сценарий

Для кандидата, который ещё не получил приглашений (Т3, критерий 6): рекрутер публикует вакансию, кандидат видит ленту с фильтрами и откликается. Отклик не заменяет основную механику: в подборке и банке кандидатов приоритет у категорий.

Вакансия (`POST/PUT /api/vacancies`): название, специализация, грейд, обязательный и желательный стек (только коды справочника, иначе `422 unknown_skills`), описание, «чем занимается команда», формат, город, **вилка от–до обязательна, от ≤ до**. Создаётся черновиком своей компании; статусы `draft → published ⇄ unpublished` (`POST /api/vacancies/{id}/publish`, `…/unpublish`, иначе `409 invalid_transition`). Рекрутер видит и меняет только вакансии своей компании (чужая — `404`).

Лента (`GET /api/vacancies` для соискателя): только опубликованные вакансии незаблокированных компаний; фильтры — специализация, грейд, формат, стек (все выбранные технологии), «зарплата до не ниже», поиск по тексту; пагинация `limit` / `offset`, всего — в заголовке `X-Total-Count`. В карточке — `my_application_id`, если кандидат уже откликнулся.

Отклик (`POST /api/vacancies/{id}/applications`, необязательное сопроводительное письмо): только на опубликованную вакансию и один раз (`409 already_applied`). **Контакты откликнувшегося открыты компании сразу** — кандидат проявил инициативу сам; раскрытие пишется в `audit_log` (`contacts_revealed`), рекрутерам компании уходит письмо с контактами, в ATS компании — вебхук `application.created` (3.13). В списке откликов рекрутер видит контакты, категорию и результат теста кандидата.

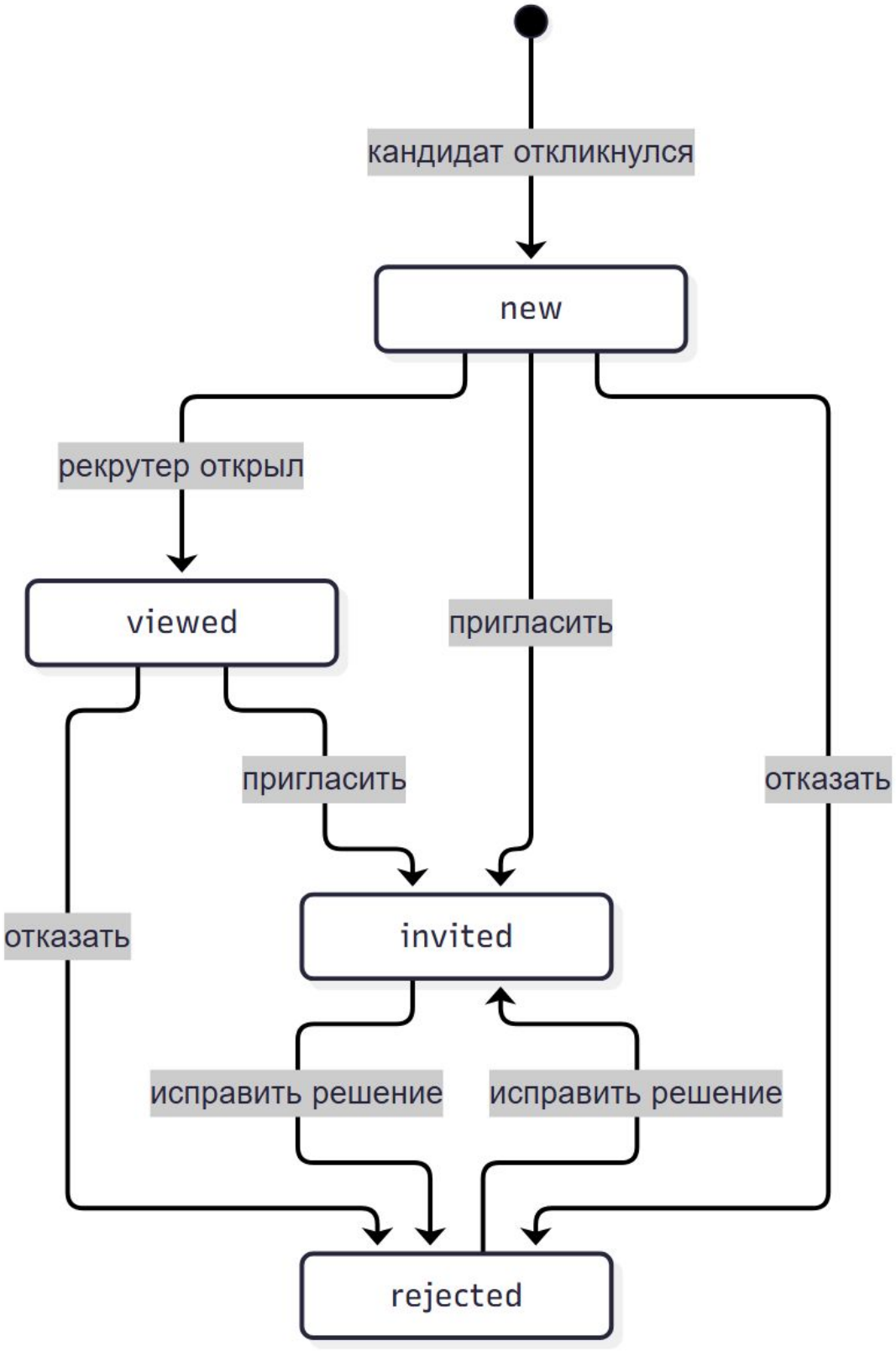


Диаграмма 8

Статус меняет рекрутер (`PATCH /api/applications/{id}`): `new → viewed → invited / rejected`; вернуться к `new` или `viewed` нельзя (`409 invalid_transition`), `invited ⇄ rejected` — исправление решения. Кандидат видит статус в своих откликах (`GET /api/applications`) и получает письмо о каждой смене («работодатель посмотрел ваш отклик», «приглашает пообщаться — ваши контакты у него есть», «выбрал другого кандидата»).

3.12. Регулярные задания и свежесть профиля

Работодатель публикует короткую задачу («решите или предложите подход») для категории (`POST /api/tasks`: название, условие, категория, срок ответа 1–30 дней, по умолчанию 7). Кандидат видит её в разделе «Задания» (`GET /api/tasks/inbox`) и отвечает (`POST /api/tasks/submissions/{id}/answer`), рекрутер оценивает ответы **обезличенно** — видит карточку «Кандидат #XXXX», а не ФИО — по шкале 0–3 с комментарием (`POST /api/tasks/submissions/{id}/grade`). Код — `backend/app/tasks/`.

Раздача. Задание получают кандидаты его категории, у которых нет активного задания и которым оно ещё не выдавалось (не заблокированы, поиск не на паузе); первыми — те, кто дольше всех был без заданий; не больше 50 за проход (`TASKS_MAX_ASSIGN_PER_RUN`). Раздача идёт сразу при создании задания, по повторной раздаче рекрутером (`POST /api/tasks/{id}/distribute`) и по расписанию. Задания заблокированных компаний не раздаются. Жизненный цикл выданного задания: `assigned → submitted → graded` или `assigned → expired` (срок вышел без ответа; ответ после срока — `409 submission_expired`). У кандидата одновременно не больше одного активного задания.

Свежесть профиля (`candidate_profiles.freshness`, 0..1) — фактор ранжирования (3.5):

Событие	Изменение свежести	Настройка
Кандидат отправил ответ	+0,08	`TASKS_FRESHNESS_ANSWER_BONUS`
Рекрутер впервые оценил ответ баллом 0–3	+0,04 + 0,03 × балл (от +0,04 до +0,13)	`TASKS_FRESHNESS_GRADE_BASE`, `TASKS_FRESHNESS_GRADE_PER_POINT`
Переоценка ответа	± 0,03 × разница баллов	—
Каждые сутки (планировщик)	× 0,5^(дней / 30) — вдвое за 30 дней; новые ответы и оценки перекрывают снижение	`TASKS_FRESHNESS_HALF_LIFE_DAYS`

Значение ограничено отрезком [0; 1]. Пример: ответ, оценённый на 3, даёт 0,08 + 0,13 = 0,21; через 30 дней без новых ответов от этого остаётся 0,105. При свежести от 0,6 в объяснении подборки появляется причина «Профиль свежий: недавно решал задания».

Периодические работы (cron или запрос администратора):

Команда (из каталога `backend/`)	Что делает	Как часто
`python -m app.tasks daily`	просрочить задания без ответа, снизить свежесть всех профилей	раз в сутки
`python -m app.tasks distribute`	раздать активные задания кандидатам без активного задания	раз в N дней, например раз в неделю
`python -m app.tasks tick`	всё сразу	—
`python -m app.integrations`	повторно отправить вебхуки ATS, у которых подошло время повтора	раз в минуту

Снижение свежести считается по времени с прошлого запуска (оно записано в журнале `freshness_decayed`), поэтому пропущенный или повторный запуск не искажает результат. То же для заданий администратор запускает через `POST /api/tasks/jobs/run?job=daily|distribute|tick`. Как поставить в cron на сервере — раздел 8.

3.13. Защита от недобросовестных работодателей и интеграция с ATS

Полная концепция (что работает, что в плане: подтверждение домена и ИНН, пробный режим для новых компаний, адаптеры к ATS) — `docs/concepts/antifraud_ats.md`.

Что уже работает в механике: обезличенный профиль до согласия кандидата, квота приглашений, обязательная вилка не ниже ожиданий кандидата, отказ с причиной.

Скоринг надёжности компании (0..100, админ видит его в разделе «Компании» вместе с составляющими; `trust_score` в `backend/app/companies/service.py`): 30 + 50 × доля приглашений, на которые кандидат ответил (принял или отклонил), среди тех, где ответ уже мог быть (нет данных — 50); +10 за подтверждённый домен почты; −15 за каждую подтверждённую жалобу; −5 за жалобу в работе; отклонённые жалобы не учитываются.

Жалобы. Соискатель жалуется на компанию (`POST /api/reports`: причина `fake_vacancy`, `spam`, `salary_mismatch`, `rude`, `other` и текст) — только если с ней был контакт (приглашение, отклик или задание), иначе `403 no_contact_with_company`; пока жалоба на рассмотрении, вторую на ту же компанию подать нельзя (`409 report_exists`). Админ ведёт очередь (`GET /api/admin/reports`): `new →`

`in_review → resolved | rejected` (`PATCH /api/admin/reports/{id}`); при подтверждении можно сразу заблокировать компанию (`block_company: true`, только вместе с `resolved`). Каждое действие — в `audit_log`.

Блокировка компании (по подтверждённой жалобе или вручную в разделе «Компании»):

Где действует	Что происходит
Приглашения	рекрутер не может отправить новое приглашение — `403 company_blocked`
Вакансии	нельзя создать вакансию и опубликовать её; уже опубликованные вакансии скрыты из ленты соискателя
Регулярные задания	нельзя создать задание и раздать его повторно — `403 company_blocked`; планировщик задания компании не раздаёт
API-токены ATS	выгрузка по токену отвечает `403 company_blocked`

Уже отправленные приглашения и полученные отклики блокировка не удаляет — история остаётся для разбора.

Интеграция с ATS (`backend/app/integrations/`, управляет администратор компании):

- **API-токен компании** — `POST /api/companies/my/api-tokens`; показывается один раз, в БД хранится только хеш SHA-256; отзыв — `DELETE …/{id}`; выпуск и отзыв — в журнале.
- **Выгрузка** — `GET /api/companies/ats/export` с заголовком `X-Api-Token`, без входа пользователя: вакансии, приглашения (контакты — только у принятых) и отклики с контактами; параметр `since` отдаёт только изменения после указанного момента.
- **Вебхуки** — `POST /api/companies/my/webhooks`: адрес и события `invitation.accepted` (кандидат принял приглашение) и `application.created` (новый отклик), плюс `ping` для проверки. Запрос — JSON `{id, event, company_id, created_at, data}` с заголовками `X-FSP-Event`, `X-FSP-Delivery`, `X-FSP-Timestamp` и `X-FSP-Signature: sha256=<hex HMAC-SHA256(секрет, "<timestamp>." + тело)>`; получатель пересчитывает подпись своим секретом и отбрасывает старые запросы по времени.
- **Надёжная доставка** — событие сначала пишется в таблицу доставок в той же транзакции, затем отправляется; не 2xx или нет связи — повтор через 1 мин, 5 мин, 30 мин, 2 ч (всего 5 попыток, потом `failed`), повторы шлёт `python -m app.integrations`. Журнал доставок — `GET /api/companies/my/webhooks/{id}/deliveries`. На проде адреса localhost и частных сетей запрещаются (`INTEGRATIONS_ALLOW_PRIVATE_URLS=false`, защита от SSRF).

3.14. Качество подбора

На 100 синтетических потребностях с эталонной разметкой (1200 кандидатов) платформа ставит в топ-10 в среднем 4,9 подходящих из 10: **precision@10 0,486** против 0,253 у поиска по ключевым словам резюме (BM25) и 0,013 у случайного порядка; **nDCG@10 0,577** против 0,313; recall@50 0,778 против 0,577. Главный вклад — категория из теста и справочник синонимов стека. Вес θ внутри категории слегка снижает соответствие эталону (0,486 против 0,511 без θ), поэтому рекомендуем вес θ 0,10 вместо 0,25 — это настройка администратора. Подробно, с абляцией и ограничениями — раздел 4.5.

Жюри может проверить подбор на своих парах: `POST /api/eval/matching` принимает CSV (`need_id`, `candidate_id`, необязательно `relevant`) и возвращает метрики.

4. Процедура валидации и результаты

Готовых датасетов и эталонной разметки в хакатоне нет [ТЗ 2], поэтому мы проверяем решение на **собственных синтетических данных, где правильный ответ известен заранее**, и на небольших наборах реальных по форме текстов (резюме, вакансии, формулировки потребностей) с ручной разметкой. Полный отчёт с графиками — `docs/validation/report.md`.

4.1. Принципы

1. **Скрытая истина.** У синтетического кандидата есть истинный уровень θ, который система не видит; она видит только его ответы, сгенерированные по модели IRT. Поэтому точность грейда можно посчитать честно.
2. **Разметка не повторяет формулу.** Эталон релевантности пар «потребность — кандидат» строится по отдельному правилу, а не по формуле ранжирования, — иначе метрика подбора проверяла бы сама себя.
3. **Настоящий код.** Симуляции прогоняют те же функции, что работают в продукте (`CATSession`, `decide`, `detect_leaks`, `calibrate_item`, `score_candidate`), а не их копии.
4. **Воспроизводимость.** Фиксированный seed, одна команда — те же цифры. Пороги ключевых метрик зашиты в автотесты: если код испортит качество, CI упадёт.
5. **Жюри может проверить само** — загрузить свои пары и получить метрики (4.6).

4.2. Что проверяем и чем — по критериям ТЗ 6.1 и 6.2

Критерий жюри	Что измеряем	Метрика	Где
Устойчивость к распространению заданий	20% банка «слито», 10% кандидатов — «читеры»; сравнение режимов без защиты / с параметризацией / с детектором	завышение грейда у читеров и честных; точность и полнота детектора задач и кандидатов	`engine.md` §9, `report.md`
Согласованность (стабильный грейд при повторе)	те же кандидаты проходят тест второй раз с другими seed	доля того же грейда, каппа Коэна, корреляция θ_1 и θ_2	`engine.md` §9, `report.md`
Дискриминативность заданий	различающая сила в адаптивном и линейном тесте	средняя a, точечно-бисериальная корреляция r_pb, доля задач с r_pb > 0,2	`engine.md` §9, `report.md`
Соответствие эталону	оценка уровня против скрытого истинного θ	корреляция θ и θ, RMSE, грейд точно и ±1	`engine.md` §9, `report.md`
Корректность категоризации	категория по тесту против истинной специализации × грейда	доля верных категорий	`report.md`
Доля релевантных в топе выдачи	100 потребностей с эталонной разметкой кандидатов	precision@10, nDCG@10, сравнение с поиском по ключевым словам резюме	`report.md`
Разбор текстов (ML/NLP — преимущество по ТЗ 3.2)	резюме, потребности, вакансии против ручной разметки	точность по полям, precision / recall стека	`docs/validation/*.md`

4.3. Данные

Набор	Объём	Как устроен	Где
Синтетические кандидаты для движка теста	1000	истинный θ ~ N(0, 1); заявленный грейд в 60% совпадает с истинным, по 20% — на грейд выше или ниже	`app/testing/engine/simulate.py`
Синтетический банк для движка	240 задач + 20 пилотных	a — логнормальное (медиана 1,3), b — равномерно от −2,8 до 2,8, 8 тем	там же
Синтетические кандидаты платформы	1200	истинный θ ~ N(0, 1) и истинный стек скрыты; заявленный грейд: 60% честно, 25% на ступень выше, 15% ниже; в резюме 15% технологий забыты, 45% написаны сленгом («постгрес», «k8s»), у 20% приписаны лишние; у 40% нет ФСП-истории	`data/validation/candidates.json`
Потребности с эталонной разметкой	100 (из 101 сгенерир ованной оставлены те,	направление, грейд, 1–3 обязательных и 0–2 желательных технологии, вилка, формат; в среднем 16,2 подходящих на потребность	`data/validation/needs.json`, `gold.csv`

Набор	Объём	Как устроен	Где
	где ≥ 3 подходящих)		
PDF-резюме	5	разная вёрстка, в т.ч. две колонки; эталон `*.expected.json`	`data/samples/resumes/`
Формулировки потребностей	12	разговорные фразы рекрутеров с эталоном полей	`data/samples/needs_phrases.json`
Вакансии	5	по одной на 5 специализаций, разметка тем	`data/samples/vacancies/`

4.4. Результаты: механика тестирования

Проверок две, и цифры в них разные, потому что разные условия:

- **Платформа на текущем банке** (4.4.1) — настоящий онбординг (тест на заявленный грейд, при неудаче — ниже, при уверенном прохождении — выше) на рабочем банке 192 шаблона (32 на направление) с настройками стенда: стоп при SE < 0,30, максимум 20 задач. Это то, что получит кандидат сегодня.
- **Движок на большом синтетическом банке** (4.4.2) — 240 задач на одно направление, стоп при SE < 0,35 или 15 задачах. Показывает, как механика работает, когда банк вырастет.

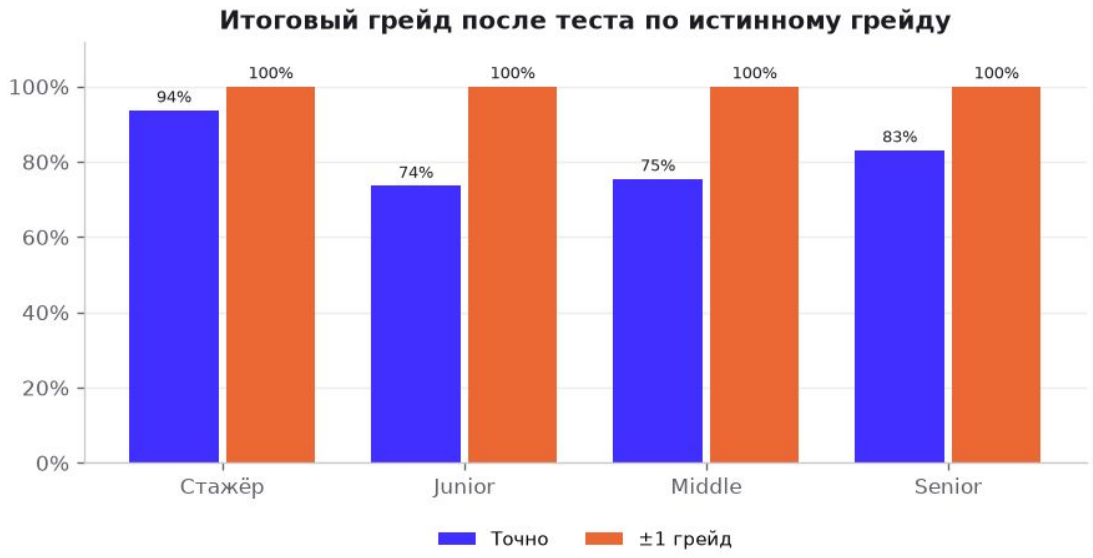
Банк в админке («Банк задач») показывает 222 записи: 192 рабочих шаблона движка и 30 демо-записей из фикстур `data/seed/items.json` (метрики и предпросмотр для показа экранов). У демо-записей нет генератора в движке, в адаптивный тест и в валидацию они не попадают.

4.4.1. Платформа на текущем банке (192 шаблона, правила стенда)

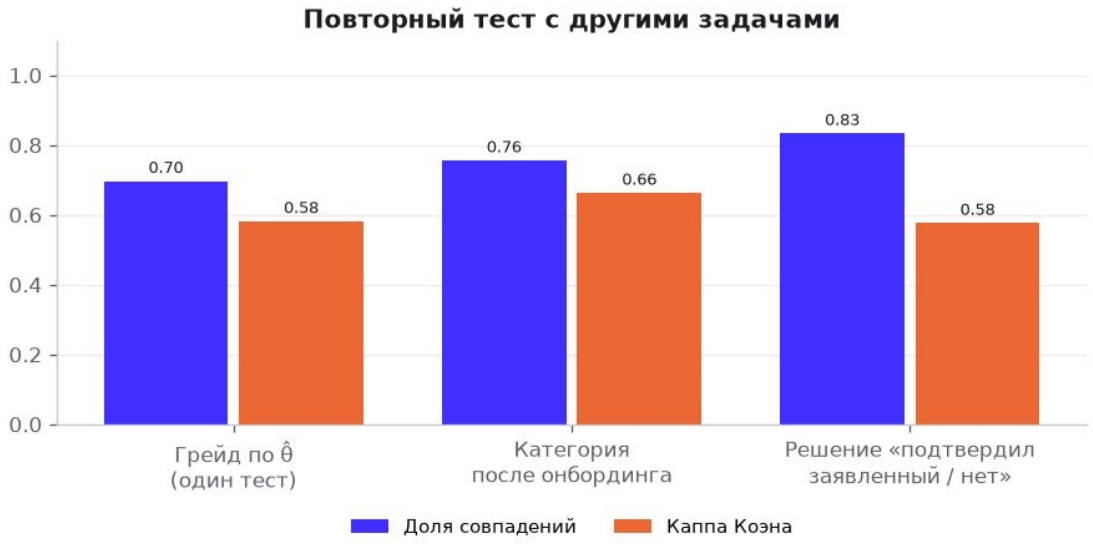
1200 синтетических кандидатов, seed 20261011, прогон 07.10.2026 (подробно — `docs/validation/report.md`, разделы 4 и 5):

Метрика	Значение
Корреляция θ первого теста с истинным θ / RMSE	0,940 / 0,358
Средняя SE в конце теста; средняя длина теста	0,374; 20,0 задачи
Грейд по θ одного теста: точно / ±1	80,2% / 99,8%
Итоговая категория после онбординга: точно / ±1	**78,8% / 100,0%**
Итоговая категория, если истинный θ дальше 0,25 от порога (57% кандидатов)	84,5%
Для сравнения: заявленный при регистрации грейд совпадает с истинным	64,9%
Завысили грейд при регистрации (281 чел.): получили истинный / остались выше	74,4% / 15,3%
Повтор онбординга с другими задачами: та же категория / каппа Коэна	75,6% / 0,663
Повтор: корреляция θ первого и второго теста	0,889
Различающая сила: средняя r_pb по 192 шаблонам / доля с r_pb > 0,2 / минимум	0,430 / 100% / 0,201
r_pb в адаптивном тесте (190 показанных шаблонов): средняя / доля > 0,2	0,374 / 91,6%
Ошибка экспертных a, b (b ± 0,3, a × e^±0,3): корреляция / грейд точно	0,941 / 78,4%

Истинный грейд	Кандидатов	Категория точно	±1
Стажёр	189	93,7%	100,0%
Junior	413	73,6%	100,0%
Middle	415	75,4%	100,0%
Senior	183	83,1%	100,0%



Итоговая категория против истинного грейда

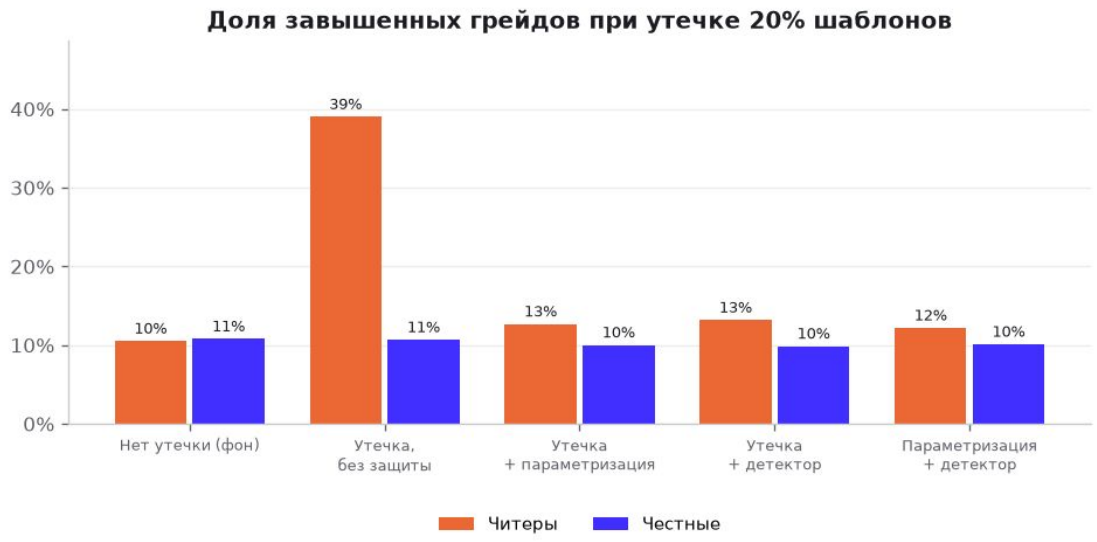


Повтор онбординга: совпадение категории

Утечка 20% банка (6000 тестов, 10% «читеров», которые на слитой задаче отвечают верно и быстро; детектор запускается по ходу потока):

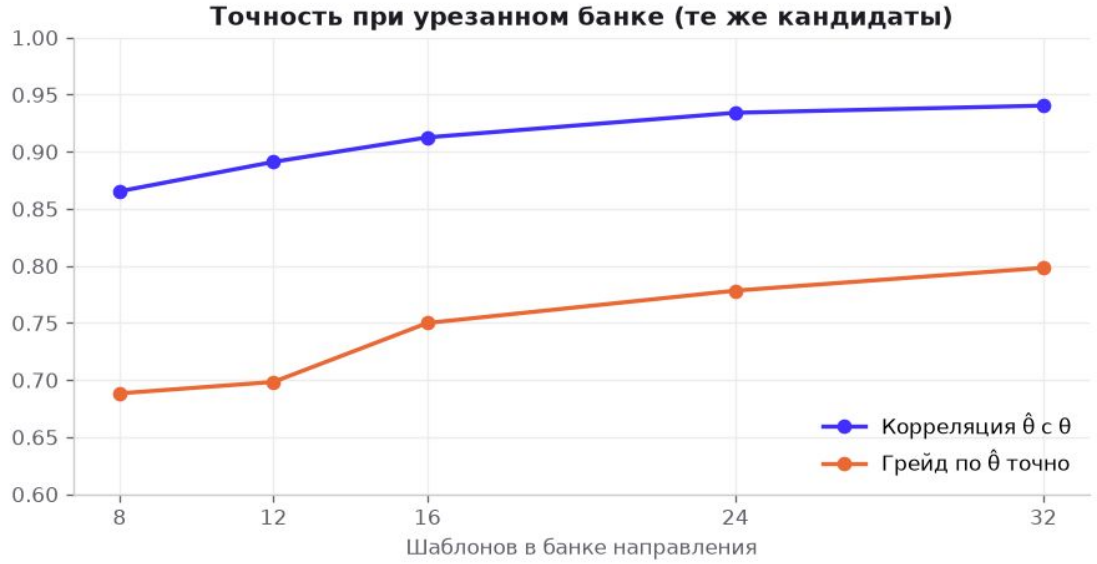
Режим	Читерам завышен грейд	Честным завышен	Детектор задач: найдено слитых / ложных
Нет утечки (фон)	10,5%	10,8%	— / 1
Утечка, без защиты	39,0%	10,7%	100% / 0
Утечка + параметризация	**12,7%**	10,0%	0% / 1
Утечка + детектор (шаблон один на всех)	13,2%	9,8%	100% / 2
Параметризация + детектор	12,2%	10,1%	3% / 4

Параметризация возвращает картину почти к фону ошибки измерения (11%): слитый ответ к чужому варианту не подходит. Если шаблон всё же один на всех, детектор находит все 36 слитых шаблонов (полнота 100%, точность 95%: 2 ложных из 156 честных) и снижает завышение до 13%. Детектор кандидатов без параметризации находит 71% читеров с точностью 99% — это повод для ручной проверки, а не наказание.



Утечка заданий и детектор

Что честно не получается на текущем банке. На 32 шаблонах в направлении цель SE < 0,30 за 20 задач недостижима (средняя итоговая SE 0,37): все тесты идут до 20 задач, тест показывает больше половины банка, а повтор даёт ту же категорию в 76% случаев. Точность ещё растёт с размером банка (8 шаблонов — грейд точно 68,8%, 16 — 75,0%, 32 — 79,8%), поэтому главный путь к стабильности — расширение банка (4.4.2 и раздел 2.13).



Точность от размера банка

4.4.2. Движок на большом синтетическом банке

Синтетическая валидация движка, seed 20261011 (подробно — `docs/testing/engine.md` §9):

Метрика	Значение
Корреляция θ с истинным θ / RMSE	0,945 / 0,332
Грейд точно / ±1	78,4% / 100%
Грейд точно, если θ дальше 0,25 от порога	88,5%
Средняя длина теста	12,8 задачи (максимум 15)
Повтор: тот же грейд / каппа Коэна / корреляция θ_1 и θ_2	67,9% / 0,558 / 0,891
Повтор: то же решение по заявленному грейду / каппа	86,4% / 0,587
Правило продукта (SE < 0,30 или 20 задач): грейд точно; повтор — тот же грейд / каппа	80,1%; 73,6% / 0,636
Средняя a; r_pb на линейной форме (доля > 0,2)	1,32; 0,348 (100%)

Метрика	Значение
Утечка без защиты: грейд завышен у читеров / у честных	85,0% / 9,9%
Утечка + параметризация: грейд завышен у читеров / у честных	14,0% / 9,8%
Детектор задач (одна задача на всех): точность / полнота	100% / 93,8%
Калибровка (30 пилотов × 300 ответов): RMSE a / b	0,198 / 0,228

Как читать: ошибка грейда больше чем на один уровень — 0%; почти все промахи — у кандидатов рядом с порогом. Стабильность «тот же грейд при повторе» ограничена длиной теста при четырёх полосах шириной в одну единицу θ; более длинный тест стабильнее — это настройка, а не переделка. Условия утечки здесь другие (другой банк, правило остановки, частота запуска детектора), поэтому проценты завышения у читеров (85% без защиты) напрямую с 4.4.1 (39%) не сравниваются; вывод одинаков: параметризация возвращает картину к фону честных кандидатов.

4.5. Результаты: подбор

100 потребностей, 1200 кандидатов. Эталон «подходит» строится только по скрытой правде: истинное направление совпадает, кандидат *реально* владеет всеми обязательными технологиями (истинный стек, а не резюме), минимальная зарплата не выше «до» вилки; оценка 2 — истинный грейд равен грейду потребности, 1 — на ступень выше (переквалифицирован), иначе 0. Эталон специально не повторяет формулу: он не видит ФСП, свежесть, формат и желательный стек, а платформа не видит истинный уровень и стек (`report.md`, раздел 3).

Платформа строит выдачу так, как её видит рекрутер: сначала основная категория потребности (категория — из теста), затем рекомендованные соседние категории, затем остальные; внутри — по счёту `score_candidate` с весами по умолчанию. В скобках — 95% бутстреп-интервал по потребностям.

Метрика	Платформа	Ключевые слова резюме (BM25)	Только категория	Случайно
precision@10	**0,486** (0,44–0,53)	0,253 (0,23–0,28)	0,216	0,013
мягкая precision@10 (оценка ≥ 1)	0,560	0,294	0,237	0,016
nDCG@10	**0,577** (0,54–0,62)	0,313 (0,28–0,34)	0,236	0,014
recall@50	0,778 (0,75–0,81)	0,577 (0,54–0,62)	0,649	0,044
Доля всех подходящих, попавших в топ-10	29,9%	15,6%	—	—

«Только категория» — кандидаты категории из теста сверху, внутри случайный порядок: показывает, сколько даёт сама категоризация без формулы счёта.

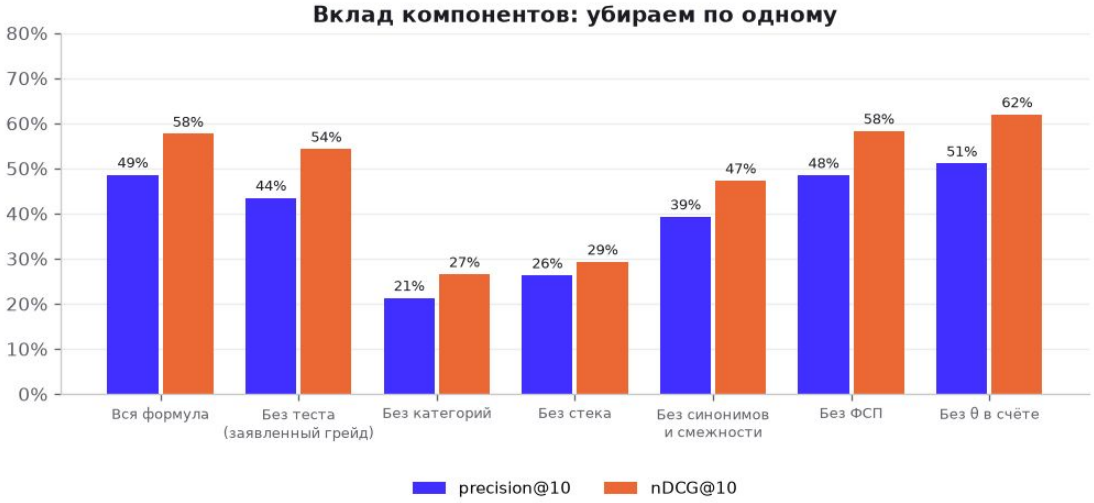
Платформа ставит в топ-10 в среднем 4,9 подходящих из 10, поиск по ключевым словам — 2,5, случайный выбор — 0,13; интервалы не пересекаются. По nDCG@10 платформа лучше ключевых слов в 87 потребностях из 100. Если все кандидаты пройдут тест заново, топ-10 совпадает в среднем на 62% (меняются кандидаты на границе категорий); при одних и тех же данных выдача детерминирована.



precision@10 и nDCG@10 по способам

Вклад компонентов (абляция).

Вариант	precision@10	nDCG@10	recall@50
Вся формула (веса по умолчанию)	0,486	0,577	0,778
Без теста: категория по заявленному грейду, θ нет	0,435	0,543	0,782
Без категорий: только счёт по всей базе	0,213	0,266	0,527
Без стека (вес 0)	0,263	0,293	0,708
Без синонимов и смежности технологий	0,393	0,473	0,732
Без ФСП (вес 0)	0,485	0,583	0,776
Без θ в счёте (вес 0; категория из теста остаётся)	0,511	0,620	0,775



Абляция формулы подбора

- **Главный вклад — тест и категории.** Без категорий precision@10 падает до 0,213; с категорией по заявленному грейду вместо теста — до 0,435: тест исправляет завышенные грейды.
- **Стек и справочник синонимов** — второй по силе: без стека 0,263, без синонимов 0,393 («постгрес» и «k8s» в резюме иначе теряются). Здесь и проигрывает поиск по ключевым словам.
- **ФСП** метрики почти не меняет (0,485): наш эталон его не учитывает. ФСП в счёте — требование ТЗ и сигнал для рекрутера; важно, что он не ухудшает выдачу.
- **θ внутри категории слегка снижает соответствие эталону:** 0,486 с θ против 0,511 без θ. Внутри категории θ поднимает самых сильных, а среди них чаще переквалифицированные — их истинный грейд на ступень выше, и эталон считает их подходящими лишь частично. Это расхождение между ТЗ («выше — сильнее подтверждённый профиль») и строгой разметкой «точно этот грейд».

Вес θ (остальные — пропорционально)	precision@10	nDCG@10
0,00	0,511	0,620
0,10	0,502	0,608
0,25 (по умолчанию)	0,486	0,577
0,40	0,449	0,532

Рекомендация: θ оставить в счёте (этого требует Т3), но снизить вес с 0,25 до 0,10 — precision@10 0,502 вместо 0,486. Веса — настройка платформы (`ranking_weights`), админ меняет их на экране «Справочники» без релиза; в демо-данных оставлены веса по умолчанию, на которых посчитаны все цифры этого раздела.

4.6. Результаты: разбор текстов

Что	Набор	Результат	Подробно
PDF-резюме → профиль	5 PDF	ФИО, контакты, город, специализация, грейд, стаж, формат, ссылки — 100%; стек — precision / recall 100% / 100%; роли — 92% / 100%; софт-скиллы — 94% / 100%	`resume_parsing.md`
Свободный текст потребности → поля	12 фраз	12 из 12 по всем полям (специализация, грейд, стек обязательный и желательный, вилка, формат, город, домен)	`need_parsing.md`
Текст вакансии → тест	5 вакансий	специализация 5/5, грейд 5/5, ожидаемые темы 25/25	`vacancy_tests.md`

Все три — в режиме правил, без ключа LLM: так результат не зависит от внешнего сервиса. Правила подбирались с оглядкой на образцы, поэтому отдельно проверены формулировки, которых в образцах нет (тест `test_unseen_phrases`).

4.7. Как воспроизвести и проверить самим

Что	Команда (из корня репозитория)
Симуляция движка теста (около минуты)	`PYTHONPATH=backend python -m app.testing.engine.simulate --seed 20261011`
Пороги движка в автотестах	`PYTHONPATH=backend pytest backend/tests/engine`
Отчёты по разбору текстов	`cd backend && python -m app.nlp.validation`
Полная валидация платформы: данные, симуляции, графики, `report.md` (~3–4 минуты, БД не нужна)	`pip install -e "backend[dev]" matplotlib`, затем `make validate` (= `python scripts/validation/run_all.py`)
Метрики и API проверки в автотестах	`pytest backend/tests/test_eval_metrics.py backend/tests/test_eval_api.py`

В интерфейсе: кабинет администратора → «Валидация» (`/admin/validation`) — графики точности грейда, стабильности повтора, различающей силы, precision@10, работы детектора утечек (`GET /api/eval/validation`).

Проверка подбора на своих данных: `POST /api/eval/matching` (или экран «Валидация») — загрузить CSV с колонками `need_id`, `candidate_id` и необязательно `relevant` (0/1 или 0/1/2 с «частично»). Порядок строк не важен: кандидатов каждой потребности ранжирует сама платформа, а метрики показывают, сколько подходящих оказалось наверху. Id — синтетические (`N001…`, `C0001…`) или реальные id потребностей и профилей стенда. Пример — `data/validation/example_pairs.csv` (6 потребностей × 40 кандидатов): precision@10 0,733, nDCG@10 0,786 против 0,617 / 0,679 у поиска по словам и 0,250 при случайном порядке.

```
curl -F file=@data/validation/example_pairs.csv
https://159-194-200-243.sslip.io/api/eval/matching
```

4.8. Ограничения валидации

- Синтетика проверяет, что механика делает то, что задумано, при известных допущениях (модель ответов 2PL, модель времени). На реальных людях пороги детектора и грейдов нужно уточнить по накопленным данным — механизмы для этого (калибровка, настраиваемые пороги) уже в продукте.
- Синтетика — не рынок: распределения уровней, стека, зарплат и поведения (завышение грейда, приписки в резюме) — наши допущения. Процедура переносится на реальные данные без изменений: нужна экспертная разметка пар, её можно загрузить через `POST /api/eval/matching`.

- Эталон подбора написан нами; правило специально не совпадает с формулой, но выбор признаков эталона (например, ФСП не считается признаком соответствия) — тоже наше решение.
- Банк из 32 шаблонов на направление ограничивает точность и стабильность теста (4.4.1).
- Образцы для разбора текстов маленькие (5–12 штук); это проверка работоспособности, а не статистическая оценка точности.

5. Схема интеграции с реестром ФСП

Организаторы намеренно не дали ни API, ни структуру данных ФСП [ТЗ 2, 4]. Поэтому мы сами решили, какие сведения о достижениях получать и как показывать работодателю, и сделали **работающую модель интеграции**: обе части ФСП подняты у нас локально и подключены к платформе по тем же протоколам, что будут в реальности. Подробный черновик для ФСП — `docs/integration/fsp.md`; код — `backend/app/fsp/`, мок реестра — `services/fsp-registry-mock/`, realm Keycloak — `infra/keycloak/realm-fsp.json`.

Часть	В реальности	На хакатоне
ФСП ID (вход и подтверждение личности)	Keycloak ФСП	Keycloak 25, realm `fsp`, клиент `fsp-talent` (public, PKCE), claim `fsp_id`
Реестр достижений	API реестра ФСП (его пока нет)	`fsp-registry-mock` — это и есть наше предложение контракта API
Платформа	—	модуль `fsp`, таблицы `fsp_links` (связь профиля с ФСП ID) и `fsp_achievements` (кэш)

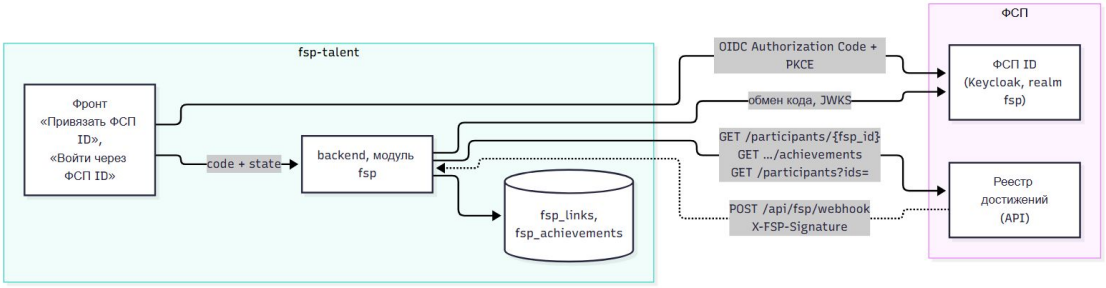


Диаграмма 9

5.1. Какие данные получаем и зачем

Работодателю важно не «участвовал ли человек», а **что это говорит о навыках**:

- **дисциплина** — продуктовое, алгоритмическое программирование, программирование систем ИБ, беспилотных авиационных систем, робототехники: к какой работе близок опыт;
- **уровень** — региональный, всероссийский, международный;
- **результат** — место, диплом I/II/III степени или участие; плюс размер зачёта («12 место из 300» и «12 место из 15» — разные вещи);
- **дата** — свежий результат весит больше старого;
- **командное или личное** участие и **роль в команде** (бэкенд, ML, капитан);
- **ссылка на протокол** — проверить результат в первоисточнике;
- **рейтинг-очки** реестра, если у ФСП есть свой рейтинг.

Не берём: дату рождения, контакты, документы, школу и вуз — для подбора не нужны (минимизация данных, 152-ФЗ). ФИО и регион — только для сверки с профилем; работодателю до принятия приглашения не показываются.

5.2. Вход и привязка через ФСП ID (OIDC)

Основной вход — свой (почта + пароль). ФСП ID нужен для «Привязать ФСП ID» (настройки кандидата) и «Войти через ФСП ID» (только для уже привязанного профиля). Протокол — OpenID Connect, Authorization Code + PKCE (S256); клиент публичный, поэтому PKCE обязателен. Номер участника приходит в токене отдельным claim `fsp_id`.

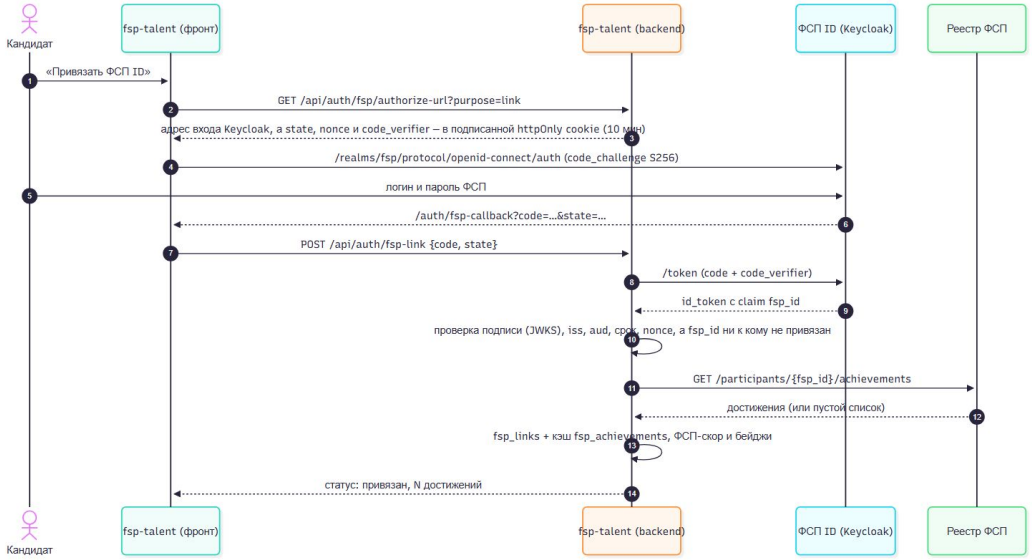


Диаграмма 10

Правило	Ответ API
ФСП ID уже привязан к другому профилю	`409 fsp_id_taken`
к профилю уже привязан другой ФСП ID	`409 fsp_already_linked`
state истёк или подменён, код не принят, ID-токен не прошёл проверку	`400 oidc_error` (причина в `details.reason`)
вход через ФСП ID, а профиль не привязан	`404 fsp_account_not_linked` — предлагаем войти по почте и привязать
Keycloak недоступен	`503 keycloak_unavailable`; вход по почте работает как обычно
реестр недоступен при привязке	привязка сохраняется, достижения подтянутся по кнопке «Обновить»
отвязка (`DELETE /api/auth/fsp-link`)	связь и кэш удаляются, ФСП-скор — «нет данных»

5.3. Контракт API реестра (наш мок)

Сервис `services/fsp-registry-mock`, Swagger — `http://localhost:18100/docs`.

Метод	Что отдаёт
`GET /health`	`{"status": "ok", "participants": 40, "synthetic": true}`
`GET /participants/{fsp_id}`	ФИО, регион, дата регистрации в ФСП, `has_history`, сводка
`GET /participants/{fsp_id}/achievements`	`{fsp_id, has_history, total, items[], updated_at}`; фильтры `discipline`, `level`
`GET /participants?ids=FSP-000001,FSP-000002`	пакетно до 100 ID: `{items[], not_found[]}` — для ночной синхронизации
`GET /dictionaries`	дисциплины (`product`, `algorithmic`, `security`, `uav`, `robotics`) и уровни (`regional`, `national`, `international`)

Коды дисциплин реестра `security` и `uav` платформа при синхронизации переводит в свои `cybersecurity` и `uas` (`DISCIPLINE_FROM_REGISTRY` в `backend/app/fsp/service.py`); остальные коды совпадают.

Ошибки — в формате платформы `{"code", "message", "details"}`: `404 participant_not_found`, `422 invalid_fsp_id`. **Участник без истории — не ошибка:** `200`, `has_history: false`, `items: []`.

Пример достижения:

```
{
  "id": "ACH-000001-05",
  "competition": {
    "id": "COMP-2025-05",
    "title": "Международный онлайн-хакатон по продуктовому программированию",
    "participants_total": 260,
    "protocol_url": "https://registry.fsp.example/protocols/COMP-2025-05.pdf"
  },
  "discipline": {"code": "product", "title": "Продуктовое программирование"},
  "level": {"code": "international", "title": "Международный"},
```

```
  "date": "2025-09-13",
  "result": {"kind": "place", "place": 1, "diploma_degree": null, "label": "1 место"},
  "participation": "team",
  "team": {"name": "Северные лисы", "size": 5, "role": "фулстек-разработчик", "is_captain":
false},
  "rating_points": 400,
  "verified": true
}
```


Тестовые данные (синтетика): `FSP-000001`–`FSP-000030` — с достижениями (от 1 до 8), `FSP-000031`–`FSP-000040` — без истории, остальные — 404.

Доступ платформы к реестру. В реальной интеграции — сервисный токен Keycloak по client credentials (отдельный конфиденциальный клиент). У нас: переменная `FSP_REGISTRY_TOKEN` на бэкенде и `FSP_MOCK_API_TOKEN` в моке — тогда все методы мока, кроме `/health`, требуют `Authorization: Bearer <токен>` (по умолчанию выключено). Клиент реестра: таймаут 5 с, 2 повтора при сетевой ошибке или 5xx (`FSP_REGISTRY_TIMEOUT_SEC`, `FSP_REGISTRY_RETRIES`).

5.4. Обновление данных

Достижения меняются редко, поэтому платформа хранит **кэш** и обновляет его:

1. **При привязке** — сразу (кандидат видит бейджи через секунду).
2. **Кнопкой «Обновить»** у кандидата — `POST /api/fsp/sync`.
3. **Вебхуком от реестра** — реестр сам сообщает об изменении (наше предложение для ФСП; на платформе реализовано).
4. **Пакетно** — `GET /participants?ids=…` пачками по 100: перезапрашиваются только участники, у которых `updated_at` новее нашей синхронизации; `not_found` → связь помечается, бейджи скрываются. Функция есть в сервисе (`FspService.sync_many`); запуск по расписанию (раз в сутки) — шаг реальной интеграции.

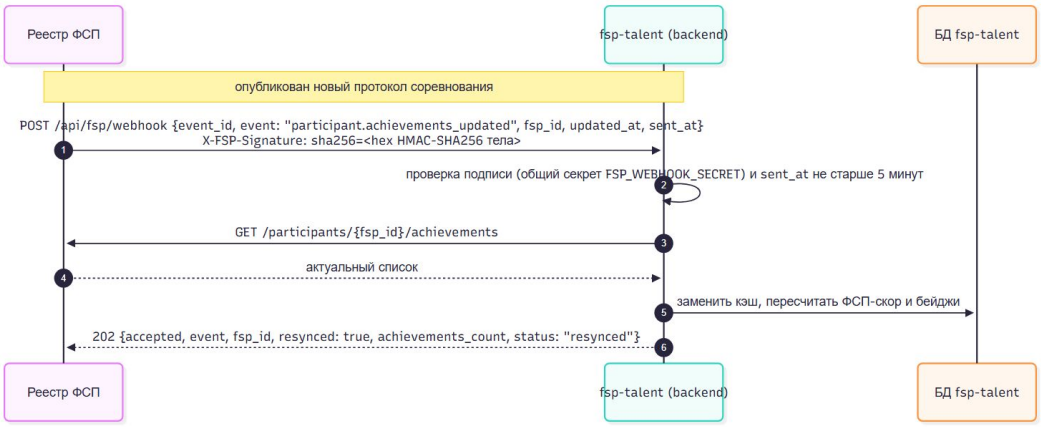


Диаграмма 11

Вебхук: тело подписывается HMAC-SHA256 общим секретом, заголовок `X-FSP-Signature: sha256=<hex>` (допускается и просто `<hex>`). Ответы: `202` — принято (для других событий и непривязанных ФСП ID — `status: "ignored"`); `401 invalid_signature`; `401 webhook_expired` (sent_at старше 5 минут); `422 validation_error`; `503 fsp_registry_unavailable` — повторите позже; `503 webhook_disabled` — секрет не задан. Повтор события безопасен: синхронизация заменяет кэш участника целиком, дублей нет.

Если реестр недоступен — показываем кэш, ничего не удаляем. Поиск, ранжирование и PDF-профиль работают на кэше.

5.5. Если истории ФСП нет

Обязательный случай по Т3. Все три ситуации обрабатываются одинаково мягко:

Ситуация	Кандидат видит	Работодатель видит	Ранжирование
ФСП ID не привязан	«Привяжите ФСП ID — достижения подтянутся автоматически»	«Нет данных ФСП» (нейтрально)	фактор ФСП = 0, штрафа нет
привязан, истории нет	«В реестре ФСП пока нет результатов»	«Нет данных ФСП»	то же

Ситуация	Кандидат видит	Работодатель видит	Ранжирование
(`has_history: false`)			
реестр недоступен	кэш	кэш	по кэшу

Отсутствие истории — не штраф. Кандидат без ФСП получает 0 по фактору «Достижения ФСП», остальные пять факторов не меняются: его место в выдаче определяют тест, стек, зарплата, формат и свежесть. В API ФСП-скор такого кандидата — `null` («нет данных»), а не 0. Формулировка «нет данных», а не «нет достижений» — чтобы не выглядело как недостаток. Демо: `candidate.nofsp@example.com` (Frontend · Junior, без истории ФСП) проходит весь путь.

5.6. Что показываем работодателю

До принятия приглашения профиль обезличен, но достижения ФСП видны — это и есть «подтверждённый профиль»:

- бейджи с пометкой **«подтверждено ФСП»** (не больше одного на уровень): «Победитель международного соревнования», «Призёр всероссийского соревнования», «Капитан команды», «Участник N соревнований ФСП»;
- результаты: дисциплина, уровень, результат («2 место из 120»), год, роль в команде;
- ФСП-скор 0–100 полоской в разбивке счёта и одна строка в «почему в подборке» («Подтверждено ФСП: призёр всероссийского соревнования по продуктовому программированию, 2026»);
- ссылка на протокол.

Не показываем до принятия: ФИО, регион, название команды (по нему легко найти человека), точные даты (только год). Кандидат может скрыть блок ФСП в настройках приватности.

5.7. ФСП-скор

Одно число 0–100 для ранжирования (вес 0,15) и объяснения (`backend/app/fsp/scoring.py`):

```
S = Σ по достижениям  L(уровень) × R(результат) × F(возраст)
ФСП-скор = round(100 × (1 - e^(-S / 3)))
```

Множитель	Значения
L — уровень	региональный 1; всероссийский 2; международный 3
R — результат	1 место 1,0; 2 место 0,85; 3 место 0,7; диплом I 0,8; диплом II 0,6; диплом III 0,45; 4–10 место 0,4; ниже 10-го 0,25; участие 0,1
F — свежесть	0,5^(возраст в годах / 2) — результат «стареет» вдвое за 2 года

Сумма с насыщением: много слабых участий не обгоняют одну победу, десятая победа добавляет меньше первой. Примеры на данных мока (на 06.10.2026): FSP-000001 (7 достижений, в т.ч. 1 место международного зачёта) — 86; FSP-000005 (лучшее — диплом III всероссийского) — 24; FSP-000007 (43 место из 48 на региональном) — 5; FSP-000031 — «нет данных».

Развитие с ФСП: множитель релевантности дисциплины специализации (ИБ — для DevOps, алгоритмическое — для Backend и Data/ML) и учёт роли в команде.

5.8. Безопасность и согласие

- Токен ФСП ID проверяется на бэкенде (подпись по JWKS realm, `iss`, `aud`, срок, `nonce`); `state` защищает от CSRF, PKCE — от перехвата кода.
- Из реестра храним только поля из 5.1. Отвязка удаляет связь и кэш.
- Вебхук — только с верной подписью и свежим `sent_at`.

5.9. Учебный стенд ФСП ID

Keycloak: локально `http://localhost:18180`, на стенде — `https://159-194-200-243.sslip.io/kc`, realm `fsp`.

Логин	ФСП ID	История
`fsp.user01` … `fsp.user05`	`FSP-000001` … `FSP-000005`	есть
`fsp.nohistory`	`FSP-000031`	нет

Пароль тестовых участников: `Fsp-demo-2026` (учебный, не секрет). Клиент `fsp-talent-test` (вход по паролю для автотестов) — только для учебного стенда, в реальной интеграции его быть не должно.

5.10. Что нужно от ФСП для реальной интеграции

1. Зарегистрировать в Keycloak ФСП клиент `fsp-talent` (public, PKCE S256, redirect URI платформы) и mapper claim `fsp_id`.
2. Выдать конфиденциальный клиент для сервисного доступа к реестру (client credentials).
3. Реализовать в реестре методы из 5.3 (или сообщить свои — меняется только `app/fsp/registry.py`).
4. По желанию — вебхук `participant.achievements_updated` с HMAC-подписью и общим секретом.

Адреса и секреты — переменные окружения (`KEYCLOAK_*`, `FSP_REGISTRY_URL`, `FSP_REGISTRY_TOKEN`, `FSP_WEBHOOK_SECRET`), код менять не нужно.

6. Описание API

6.1. Где смотреть

Что	Где
Swagger UI (можно вызывать методы)	`/api/docs` — на стенде https://159-194-200-243.sslip.io/api/docs, локально http://localhost:18000/api/docs
ReDoc	`/api/redoc`
Схема OpenAPI 3.1 (живая)	`/api/openapi.json`
Выгрузка схемы в репозитории	`docs/api/openapi.json`
Таблица методов по ролям и каталог кодов ошибок	приложение `api_endpoints.md` (генерируется скриптом)

Схему генерирует сам FastAPI из кода: у каждого метода описаны параметры, модель запроса и ответа с примерами и **все коды ошибок** с примерами тел. Выгрузка в файл — `python scripts/export_openapi.py` (или `make openapi`); CI проверяет, что файл совпадает с кодом (`export_openapi.py --check`). По этой же схеме фронтенд генерирует TS-типы (`npm run gen:api`) — контракт один на обе стороны.

Таблица в приложении строится скриптом из `openapi.json` и кода роутеров:

```
python docs/solution/tools/gen_api_table.py          # обновить docs/solution/api_endpoints.md
python docs/solution/tools/gen_api_table.py --check  # проверить, что таблица актуальна
```

6.2. Общие правила

- Префикс всех методов — `/api`. Формат — JSON (загрузка резюме — `multipart/form-data`, PDF-профиль и CSV — файлы).
- Ключи JSON, коды справочников и коды ошибок — по-английски; тексты для людей — по-русски.
- Справочные коды фиксированы: специализации `backend`, `frontend`, `mobile`, `data_ml`, `qa`, `devops`; грейды `intern`, `junior`, `middle`, `senior`; категория — `"<spec>:<grade>"`; роли `candidate`, `recruiter`, `admin`; ФСП ID — `FSP-000001`.
- Зарплата всегда диапазоном `salary_from` / `salary_to` в рублях, «от» ≤ «до».
- Даты и время — ISO 8601.
- Списки банка кандидатов — с пагинацией `page`, `page_size`.

6.3. Группы методов

Всего 130 методов (на 07.10.2026; актуальное число — в приложении).

Группа (тег)	Префикс	Для кого	Что внутри
`system`	`/api/health`	все	живость сервиса и БД, версия миграций
`auth`	`/api/auth`	все	регистрация, подтверждение почты, вход, refresh, выход, сброс пароля, `me`, вход и привязка через ФСП ID
`users`	`/api/users/me/consents`	вошедшие	согласия на обработку ПДн и публикацию профиля
`catalog`	`/api/catalog`	все	специализации, грейды, категории, темы, стек, причины отказа
`candidates`	`/api/candidates`	соискатель; рекрутер (банк)	свой профиль, резюме (PDF → поля), PDF-профиль, настройки приватности, грейд; банк кандидатов и обезличенный профиль для рекрутера
`companies`	`/api/companies/my`, `/api/companies/ats/export`	рекрутер; ATS компании по API-токену	профиль компании, рекрутеры, код приглашения, дашборд; API-токены и вебхуки ATS (администратор компании), выгрузка для ATS
`testing`	`/api/testing`	соискатель	опрос, сессии теста, ответы, результат, статус грейда и кулдауны
`fsp`	`/api/fsp`	соискатель; реестр ФСП	статус привязки, достижения, «Обновить», вебхук реестра
`matching`	`/api/matching/needs`	рекрутер	потребности, разбор свободного текста, рекомендованные категории, подборка со счётом
`invitations`	`/api/invitations`	соискатель,	приглашения, квота, счётчики, рыночная подсказка,

Группа (тег)	Префикс	Для кого	Что внутри
		рекрутер	принять / отклонить / отозвать
`vacancies`	`/api/vacancies`, `/api/applications`	соискатель, рекрутер	вакансии, публикация, отклики и их статусы
`tasks`	`/api/tasks`	соискатель, рекрутер; админ (периодические работы)	регулярные задания, раздача, ответы, оценка 0–3, `jobs/run`
`admin`	`/api/admin`, `/api/reports`	админ (жалоба — соискатель)	дашборд, пользователи, компании, справочники и настройки, банк задач (метрики, утечки, калибровка), тесты из вакансий, жалобы, журнал
`eval`	`/api/eval`	все (данные синтетические)	графики валидации, проверка подбора по CSV

6.4. Авторизация

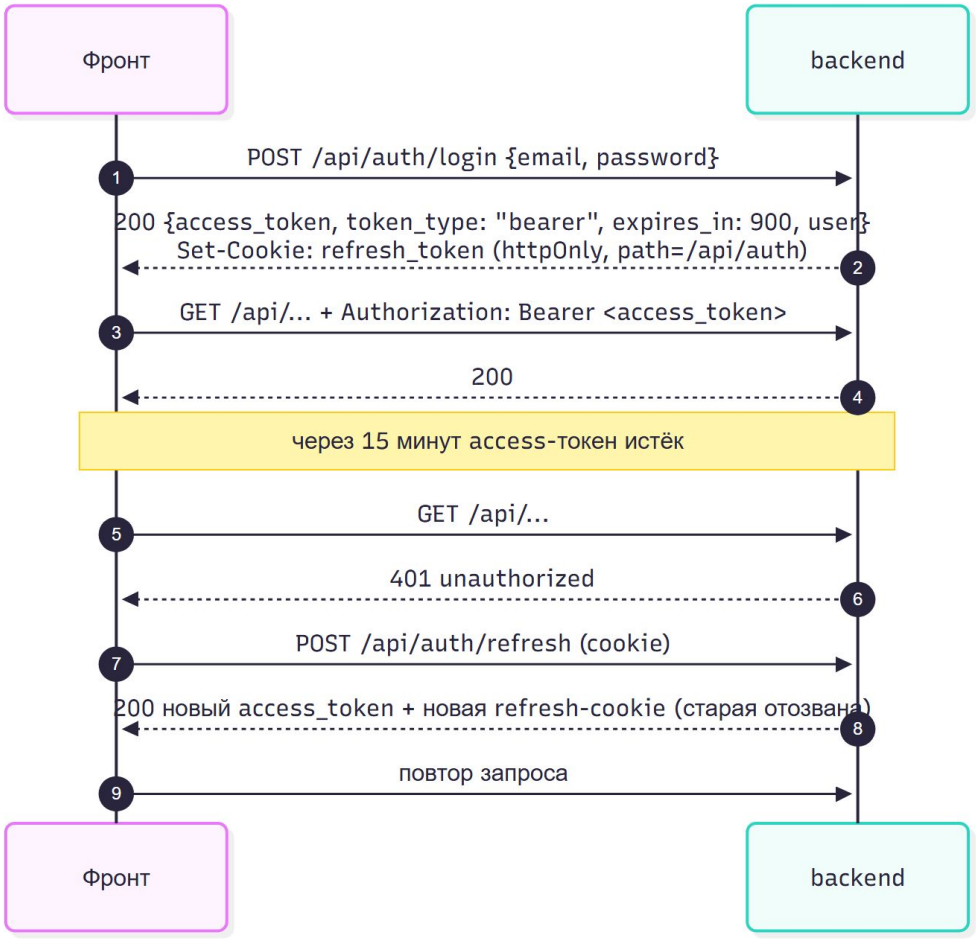


Диаграмма 12

Шаг	Метод	Детали
Регистрация	`POST /api/auth/register`	роль `candidate` или `recruiter` (рекрутер создаёт компанию или вступает по коду приглашения), почта, пароль, обязательное согласие на обработку ПДн → письмо со ссылкой (24 ч)
Подтверждение почты	`POST /api/auth/verify-email`	токен из письма; без подтверждения вход отвечает `403 email_not_verified`; повтор письма — `POST /api/auth/resend-verification` (не чаще раза в минуту)
Вход	`POST /api/auth/login`	access-токен JWT (HS256, 15 мин) в теле; refresh-токен — в httpOnly cookie `refresh_token` на пути `/api/auth` (30 дней, в БД — только хеш); 5 неудачных попыток за 15 минут на пару «почта + IP» → 429

Шаг	Метод	Детали
		too_many_attempts
Запросы	любой защищённый метод	заголовок Authorization: Bearer <access_token>; без него — 401 unauthorized, с чужой ролью — 403 forbidden, заблокирован — 403 user_blocked
Обновление	POST /api/auth/refresh	по cookie; refresh-токен ротируется — старый перестаёт действовать
Выход	POST /api/auth/logout	отзывает refresh-сессию и удаляет cookie
Сброс пароля	POST /api/auth/password-reset/request, …/confirm	письмо со ссылкой (60 мин); после смены пароля все refresh-сессии отзываются
ФСП ID	GET /api/auth/fsp/authorize-url, POST /api/auth/fsp/login, POST/DELETE /api/auth/fsp-link	OIDC + PKCE через Keycloak, см. раздел 5

В Swagger: «Authorize» → схема HTTPBearer → вставить access_token из ответа /api/auth/login.

Демо-заголовки (только dev). При DEMO_AUTH_ENABLED=true вместо токена можно передать X-Demo-Role: candidate | recruiter | admin и необязательно X-Demo-User-Id: <id> — удобно для отладки и автотестов. На стенде (infra/stand/compose.yml) заглушка выключена (DEMO_AUTH_ENABLED=false), работает только JWT.

Вебхук реестра ФСП (POST /api/fsp/webhook) аутентифицируется не токеном, а подписью тела: X-FSP-Signature: sha256=<hex HMAC-SHA256> общим секретом FSP_WEBHOOK_SECRET.

Выгрузка для ATS (GET /api/companies/ats/export) — без входа пользователя, по API-токену компании в заголовке X-Api-Token (в Swagger — схема APIKeyHeader). Токен выпускает администратор компании (POST /api/companies/my/api-tokens), он показывается один раз. **Исходящие вебхуки в ATS** платформа подписывает сама: X-FSP-Signature: sha256=<hex HMAC-SHA256(секрет вебхука, "<X-FSP-Timestamp>." + тело)> (раздел 3.13).

6.5. Формат ошибок

Любая ошибка — одно и то же тело:

```
{"code": "salary_below_expectations",
 "message": "Вилка ниже ожиданий кандидата. Поднимите верхнюю границу.",
 "details": {}}
```

HTTP	Типовые code	Когда
400	bad_request, token_invalid, oidc_error	неверный запрос, ссылка из письма, ошибка входа через ФСП ID
401	unauthorized, invalid_credentials, invalid_refresh_token, invalid_signature	нет или неверный токен, пароль, подпись
403	forbidden, user_blocked, email_not_verified, company_blocked, no_contact_with_company	чужая роль, блокировка пользователя или компании, неподтверждённая почта, жалоба без контакта с компанией
404	not_found (details.resource, details.id)	объекта нет **или он чужой** — чужие приглашения, отклики, потребности не раскрываются даже фактом существования
409	invalid_transition, quota_exceeded, already_invited, already_applied, cooldown_active, fsp_id_taken, email_taken, report_exists, …	конфликт с состоянием (статус, квота, кулдаун, уже существует)
410	token_expired	ссылка из письма истекла
413 / 415	file_too_large, unsupported_file	PDF-резюме больше 5 МБ или не PDF
422	validation_error (details.errors — список полей), salary_below_expectations, unknown_skills, …	ошибка данных; для «от» > «до» в details.errors — salary_range_invalid
429	too_many_attempts, too_many_requests	лимит попыток входа, частые письма
503	keycloak_unavailable, fsp_registry_unavailable, webhook_disabled	внешний сервис недоступен (основная работа не страдает)

Полный каталог кодов с сообщениями и числом методов — в приложении.

6.6. Примеры

Вход и подборка (стенд или локально, демо-учётка рекрутера):

```
API=http://localhost:18000/api          # или https://159-194-200-243.sslip.io/api
TOKEN=$(curl -s -H 'Content-Type: application/json' \
  -d '{"email":"recruiter@example.com","password":"demo1234"}' \
  $API/auth/login | python -c "import sys,json;print(json.load(sys.stdin)['access_token'])")

curl -s -H "Authorization: Bearer $TOKEN" $API/matching/needs
curl -s -H "Authorization: Bearer $TOKEN" "$API/matching/needs/2/candidates"
curl -s -H "Authorization: Bearer $TOKEN" \
  "$API/invitations/market-hint?category=backend:senior&salary_from=300000&salary_to=400000"
```

Без токена — `401 {"code":"unauthorized","message":"Требуется вход","details":{}}`; рекрутер на методе админа — `403 {"code":"forbidden","message":"Недостаточно прав", "details": {"required_roles":["admin"],"role":"recruiter"}}`.

6.7. API мока реестра ФСП

Отдельный сервис со своим Swagger (`http://localhost:18100/docs`): `GET /health`, `GET /participants/{fsp_id}`, `GET /participants/{fsp_id}/achievements`, `GET /participants?ids=…`, `GET /dictionaries` — см. раздел 5.3.

Приложение к разделу 6. Эндпоинты API по ролям

Источник — `docs/api/openapi.json` (fsp-talent API 0.1.0, OpenAPI 3.1.0); роль — по зависимостям в коде роутеров `backend/app/**/router.py`. Подробные схемы запросов и ответов, примеры и описания кодов ошибок — в Swagger `/api/docs`.

Всего методов: **130**.

Кто вызывает	Методов
без входа	20
реестр ФСП (HMAC-подпись)	1
ATS компании (X-Api-Token)	1
любой вошедший	3
соискатель	35
рекрутер	44
админ	32

Метод, доступный двум ролям, учтён в обеих строках. «Без входа» — публичные методы (регистрация, вход, справочники, health). «Любой вошедший» — нужен JWT, роль не важна. «ATS компании» — без входа пользователя, по API-токену компании в заголовке `X-Api-Token`. Метод с двумя ролями отдаёт каждой свою проекцию данных (например, приглашения: кандидат — полученные, рекрутер — своей компании).

Служебное (`system`)

Метод	Путь	Кто	Что делает	Коды ответов
GET	`/api/health`	без входа	Проверка живости (с проверкой БД)	200, 503

Регистрация, вход, ФСП ID (`auth`)

Метод	Путь	Кто	Что делает	Коды ответов
POST	`/api/auth/register`	без входа	Регистрация по почте	201, 404, 409, 422
POST	`/api/auth/verify-email`	без входа	Подтвердить почту по токену из письма	200, 400, 410, 422
POST	`/api/auth/resend-verification`	без входа	Отправить письмо повторно	202, 422, 429
POST	`/api/auth/login`	без входа	Вход по почте и паролю	200, 401, 403, 422, 429
POST	`/api/auth/refresh`	без входа	Обновить access-токен по refresh-cookie	200, 401, 403, 422
POST	`/api/auth/logout`	без входа	Выход	204, 422
GET	`/api/auth/me`	любой вошедший	Текущий пользователь	200, 401, 422
POST	`/api/auth/password-reset/request`	без входа	Запросить сброс пароля	202, 422, 429
POST	`/api/auth/password-reset/confirm`	без входа	Задать новый пароль по токену	200, 400, 410, 422
GET	`/api/auth/fsp/authorize-url`	без входа	Ссылка на вход/привязку через ФСП ID	200, 422, 503
POST	`/api/auth/fsp/login`	без входа	Вход через ФСП ID (обмен кода Keycloak на сессию)	200, 400, 404, 422, 503
POST	`/api/auth/fsp-link`	соискатель	Привязать ФСП ID к профилю	200, 400, 401, 403, 404, 409, 422, 503
DELETE	`/api/auth/fsp-link`	соискатель	Отвязать ФСП ID	200, 401, 403, 404, 409, 422

Согласия 152-ФЗ (users)

Метод	Путь	Кто	Что делает	Коды ответов
GET	/api/users/me/consents	любой вошедший	Мои согласия: версия текста и даты	200, 401, 422
PUT	/api/users/me/consents/{consent_type}	любой вошедший	Дать или отозвать согласие	200, 401, 409, 422

Справочники (catalog)

Метод	Путь	Кто	Что делает	Коды ответов
GET	/api/catalog/specializations	без входа	Специализации	200
GET	/api/catalog/grades	без входа	Грейды	200
GET	/api/catalog/categories	без входа	Категории (специализация × грейд)	200, 422
GET	/api/catalog/topics	без входа	Темы теста	200, 422
GET	/api/catalog/skills	без входа	Справочник стека	200, 422
GET	/api/catalog/decline-reasons	без входа	Причины отказа от приглашения	200

Кабинет соискателя и банк кандидатов (candidates)

Метод	Путь	Кто	Что делает	Коды ответов
GET	/api/candidates/me/dashboard	соискатель	Главная соискателя	200, 401, 403, 404, 422
GET	/api/candidates/me/profile	соискатель	Свой полный профиль	200, 401, 403, 404, 422
PUT	/api/candidates/me/profile	соискатель	Сохранить профиль	200, 401, 403, 404, 422
POST	/api/candidates/me/resume	соискатель	Загрузить PDF-резюме и получить предзаполнение	200, 401, 403, 404, 413, 415, 422
GET	/api/candidates/me/profile.pdf	соискатель	Скачать стандартизированный PDF-профиль	200, 401, 403, 404, 422
GET	/api/candidates/me/settings	соискатель	Настройки: приватность, согласия, ФСП ID	200, 401, 403, 404, 422
PUT	/api/candidates/me/settings/privacy	соискатель	Что видно работодателю до принятия приглашения	200, 401, 403, 404, 422
PUT	/api/candidates/me/settings/job-search	соискатель	Пауза поиска («не ищу работу»)	200, 401, 403, 404, 422
PUT	/api/candidates/me/settings/publication	соискатель	Согласие на публикацию профиля работодателям	200, 401, 403, 404, 422
GET	/api/candidates/me/grade	соискатель	Категория и грейд: статус, история, доступность теста ниже/выше	200, 401, 403, 404, 422
GET	/api/candidates	рекрутер	Банк кандидатов (обезличенно)	200, 401, 403, 422
GET	/api/candidates/{candidate_id}	рекрутер	Обезличенный профиль кандидата	200, 401, 403, 404, 422

Компания рекрутера (companies)

Метод	Путь	Кто	Что делает	Коды ответов
GET	/api/companies/my	рекрутер	Профиль моей компании	200, 401, 403, 404, 422
PUT	/api/companies/my	рекрутер	Изменить профиль компании	200, 401, 403, 404, 422
POST	/api/companies/my/invite-code	рекрутер	Выпустить новый код приглашения рекрутеров	200, 401, 403, 404, 422
GET	/api/companies/my/recruiters	рекрутер	Рекрутеры компании	200, 401, 403, 404, 422
GET	/api/companies/my/dashboard	рекрутер	Главная рекрутера	200, 401, 403, 404, 422
GET	/api/companies/my/api-tokens	рекрутер	API-токены компании для ATS	200, 401, 403, 422
POST	/api/companies/my/api-tokens	рекрутер	Выпустить API-токен	201, 401, 403, 422
DELETE	/api/companies/my/api-	рекрутер	Отозвать API-токен	204, 401, 403,

Метод	Путь	Кто	Что делает	Коды ответов
	tokens/{token_id}			404, 422
GET	/api/companies/my/webhooks	рекрутер	Вебхуки компании	200, 401, 403, 422
POST	/api/companies/my/webhooks	рекрутер	Добавить вебхук	201, 401, 403, 422
DELETE	/api/companies/my/webhooks/{webhook_id}	рекрутер	Удалить вебхук	204, 401, 403, 404, 422
POST	/api/companies/my/webhooks/{webhook_id}/ping	рекрутер	Проверить вебхук: отправить событие ping	200, 401, 403, 404, 422
GET	/api/companies/my/webhooks/{webhook_id}/deliveries	рекрутер	Журнал доставок вебхука	200, 401, 403, 404, 422
GET	/api/companies/ats/export	ATS компании (X-Api-Token)	Выгрузка данных компании для ATS (по API-токену)	200, 401, 403, 422

Опрос, адаптивный тест, грейд (testing)

Метод	Путь	Кто	Что делает	Коды ответов
GET	/api/testing/survey	соискатель	Вопросы опроса по специализации	200, 401, 403, 422
POST	/api/testing/survey	соискатель	Отправить опрос → рекомендованная специализация и темы	200, 401, 403, 422
GET	/api/testing/grade	соискатель	Категория и грейд: статус, кулдауны, история	200, 401, 403, 404, 422
GET	/api/testing/sessions	соискатель	Мои тесты	200, 401, 403, 422
POST	/api/testing/sessions	соискатель	Начать тест	201, 401, 403, 409, 422
GET	/api/testing/sessions/{session_id}	соискатель	Экран теста: текущая задача, прогресс, таймер	200, 401, 403, 404, 422
POST	/api/testing/sessions/{session_id}/answers	соискатель	Ответить на текущую задачу	200, 401, 403, 404, 409, 422
GET	/api/testing/sessions/{session_id}/result	соискатель	Результат теста: категория, θ и интервал, разбор по темам	200, 401, 403, 404, 409, 422

Достижения ФСП (fsp)

Метод	Путь	Кто	Что делает	Коды ответов
GET	/api/fsp/status	соискатель	Статус привязки ФСП ID, ФСП-скор и бейджи	200, 401, 403, 404, 422
GET	/api/fsp/achievements	соискатель	Достижения ФСП (подтверждено ФСП)	200, 401, 403, 404, 422
POST	/api/fsp/sync	соискатель	Обновить достижения из реестра ФСП	200, 401, 403, 409, 422, 503
POST	/api/fsp/webhook	реестр ФСП (HMAC-подпись)	Вебхук реестра ФСП: достижения участника изменились	202, 401, 422, 503

Потребности и подборка (matching)

Метод	Путь	Кто	Что делает	Коды ответов
POST	/api/matching/needs/parse	рекрутер	Разобрать свободный текст потребности в поля	200, 401, 403, 422
GET	/api/matching/needs	рекрутер	Потребности моей компании	200, 401, 403, 422
POST	/api/matching/needs	рекрутер	Создать потребность	201, 401, 403, 422
GET	/api/matching/needs/{need_id}	рекрутер	Потребность	200, 401, 403, 404, 422
PUT	/api/matching/needs/{need_id}	рекрутер	Изменить потребность	200, 401, 403, 404, 422
DELETE	/api/matching/needs/{need_id}	рекрутер	Удалить потребность	204, 401, 403, 404, 422
GET	/api/matching/needs/{need_id}/categories	рекрутер	Рекомендованные категории с числом кандидатов	200, 401, 403, 404, 422
GET	/api/matching/needs/{need_id}/candidates	рекрутер	Подборка: кандидаты категории с объяснением и разбивкой счёта	200, 401, 403, 404, 422

Приглашения и выход на контакт (`invitations`)

Метод	Путь	Кто	Что делает	Коды ответов
GET	`/api/invitations`	соискатель, рекрутер	Мои приглашения	200, 401, 403, 422
POST	`/api/invitations`	рекрутер	Отправить приглашение	201, 401, 403, 404, 409, 422
GET	`/api/invitations/quota`	рекрутер	Квота активных приглашений рекрутера	200, 401, 403, 422
GET	`/api/invitations/counters`	соискатель, рекрутер	Счётчики для бейджей кабинета	200, 401, 403, 422
GET	`/api/invitations/market-hint`	рекрутер	Рыночная подсказка зарплаты по категории	200, 401, 403, 404, 422
GET	`/api/invitations/{invitation_id}`	соискатель, рекрутер	Карточка приглашения с журналом событий	200, 401, 403, 404, 422
POST	`/api/invitations/{invitation_id}/accept`	соискатель	Принять приглашение (кандидат)	200, 401, 403, 404, 409, 422
POST	`/api/invitations/{invitation_id}/decline`	соискатель	Отклонить приглашение с причиной (кандидат)	200, 401, 403, 404, 409, 422
POST	`/api/invitations/{invitation_id}/withdraw`	рекрутер	Отозвать приглашение (рекрутер)	200, 401, 403, 404, 409, 422

Вакансии и отклики (`vacancies`)

Метод	Путь	Кто	Что делает	Коды ответов
GET	`/api/vacancies`	соискатель, рекрутер	Вакансии	200, 401, 403, 422
POST	`/api/vacancies`	рекрутер	Создать вакансию (черновик)	201, 401, 403, 422
GET	`/api/vacancies/{vacancy_id}`	соискатель, рекрутер	Карточка вакансии	200, 401, 403, 404, 422
PUT	`/api/vacancies/{vacancy_id}`	рекрутер	Изменить вакансию своей компании	200, 401, 403, 404, 422
POST	`/api/vacancies/{vacancy_id}/publish`	рекрутер	Опубликовать вакансию	200, 401, 403, 404, 409, 422
POST	`/api/vacancies/{vacancy_id}/unpublish`	рекрутер	Снять вакансию с публикации	200, 401, 403, 404, 409, 422
GET	`/api/vacancies/{vacancy_id}/applications`	рекрутер	Отклики на вакансию (рекрутер)	200, 401, 403, 404, 422
POST	`/api/vacancies/{vacancy_id}/applications`	соискатель	Откликнуться на вакансию (кандидат)	201, 401, 403, 404, 409, 422
GET	`/api/applications`	соискатель, рекрутер	Отклики	200, 401, 403, 422
PATCH	`/api/applications/{application_id}`	рекрутер	Сменить статус отклика (рекрутер)	200, 401, 403, 404, 409, 422

Регулярные задания (`tasks`)

Метод	Путь	Кто	Что делает	Коды ответов
GET	`/api/tasks/inbox`	соискатель	Мои задания: активное и история (кандидат)	200, 401, 403, 404, 422
POST	`/api/tasks/submissions/{submission_id}/answer`	соискатель	Ответить на задание (кандидат)	200, 401, 403, 404, 409, 422
GET	`/api/tasks`	рекрутер	Задания моей компании (рекрутер)	200, 401, 403, 422
POST	`/api/tasks`	рекрутер	Создать короткое задание (рекрутер)	201, 401, 403, 422
POST	`/api/tasks/jobs/run`	админ	Запустить периодическую работу заданий (админ)	200, 401, 403, 422
GET	`/api/tasks/{task_id}/submissions`	рекрутер	Ответы кандидатов на задание (обезличенно)	200, 401, 403, 404, 422
POST	`/api/tasks/{task_id}/distribute`	рекрутер	Раздать задание ещё раз (рекрутер)	200, 401, 403, 404, 409, 422

Метод	Путь	Кто	Что делает	Коды ответов
POST	/api/tasks/submissions/{submission_id}/grade	рекрутер	Оценить ответ 0–3 (рекрутер)	200, 401, 403, 404, 409, 422

Кабинет администратора, жалобы (admin)

Метод	Путь	Кто	Что делает	Коды ответов
GET	/api/admin/dashboard	админ	Дашборд платформы	200, 401, 403, 422
GET	/api/admin/users	админ	Пользователи: поиск и фильтры	200, 401, 403, 422
PATCH	/api/admin/users/{user_id}	админ	Сменить роль или заблокировать пользователя	200, 401, 403, 404, 409, 422
GET	/api/admin/users/{user_id}/audit	админ	Журнал действий по пользователю	200, 401, 403, 404, 422
GET	/api/admin/companies	админ	Компании со скорингом надёжности	200, 401, 403, 422
PATCH	/api/admin/companies/{company_id}	админ	Заблокировать / разблокировать компанию	200, 401, 403, 404, 422
GET	/api/admin/settings	админ	Настройки: пороги θ, кулдауны, квота, веса ранжирования	200, 401, 403, 422
PUT	/api/admin/settings	админ	Изменить настройки платформы	200, 401, 403, 422
POST	/api/admin/catalog/skills	админ	Добавить технологию в справочник стека	201, 401, 403, 409, 422
PUT	/api/admin/catalog/skills/{skill_id}	админ	Изменить технологию	200, 401, 403, 404, 422
DELETE	/api/admin/catalog/skills/{skill_id}	админ	Удалить технологию	204, 401, 403, 404, 409, 422
POST	/api/admin/catalog/specializations	админ	Вернуть специализацию в справочник (код из фиксированного списка)	201, 401, 403, 409, 422
PATCH	/api/admin/catalog/specializations/{spec_id}	админ	Изменить подпись или активность специализации (код фиксирован)	200, 401, 403, 404, 422
DELETE	/api/admin/catalog/specializations/{spec_id}	админ	Удалить специализацию	204, 401, 403, 404, 409, 422
PATCH	/api/admin/catalog/grades/{grade_id}	админ	Изменить подпись грейда (код фиксирован)	200, 401, 403, 404, 422
PATCH	/api/admin/catalog/categories/{category_id}	админ	Изменить подпись категории	200, 401, 403, 404, 422
POST	/api/admin/catalog/topics	админ	Добавить тему теста	201, 401, 403, 404, 409, 422
PATCH	/api/admin/catalog/topics/{topic_id}	админ	Изменить подпись темы	200, 401, 403, 404, 422
DELETE	/api/admin/catalog/topics/{topic_id}	админ	Удалить тему	204, 401, 403, 404, 409, 422
GET	/api/admin/reports	админ	Очередь жалоб на работодателей	200, 401, 403, 422
PATCH	/api/admin/reports/{report_id}	админ	Обработать жалобу	200, 401, 403, 404, 409, 422
GET	/api/admin/audit	админ	Журнал действий	200, 401, 403, 422
GET	/api/admin/items	админ	Банк задач с метриками	200, 401, 403, 422
GET	/api/admin/items/coverage	админ	Покрытие банка: темы × грейды	200, 401, 403, 422
POST	/api/admin/items/recalibrate	админ	Пересчитать калибровку a, b по накопленным ответам	200, 401, 403, 409, 422
POST	/api/admin/items/leak-scan	админ	Запустить детектор утечек	200, 401, 403, 422
GET	/api/admin/items/{item_id}	админ	Шаблон: метрики и предпросмотр 3 вариантов	200, 401, 403, 404, 422
POST	/api/admin/items/{item_id}/retire	админ	Вывести шаблон из оборота	200, 401, 403, 404, 409, 422
POST	/api/admin/items/{item_id}/approve	админ	Одобрить черновик → пилот (непроверочный)	200, 401, 403, 404, 409, 422
GET	/api/admin/vacancy-tests	админ	Тесты, собранные из вакансий	200, 401, 403, 422
POST	/api/admin/vacancy-tests	админ	Текст вакансии → темы → тест с	200, 401, 403, 422

Метод	Путь	Кто	Что делает	Коды ответов
			объяснением	
POST	/api/reports	соискатель	Пожаловаться на работодателя (соискатель)	201, 401, 403, 404, 409, 422

Валидация и метрики (eval)

Метод	Путь	Кто	Что делает	Коды ответов
GET	/api/eval/validation	без входа	Все графики валидации	200
GET	/api/eval/charts/{chart_id}	без входа	Один график валидации	200, 404, 422
POST	/api/eval/matching	без входа	Загрузить CSV пар «потребность — кандидат» → метрики подбора	200, 413, 422

Коды ошибок

Тело любой ошибки: `{"code": "...", "message": "...", "details": {...}}`. `code` — машинный код (англ.), `message` — текст для пользователя (рус.), `details` — подробности (поля с ошибками, id объекта, дата, когда действие станет доступно).

HTTP	code	Пример сообщения	Методов
400	oidc_error	Ошибка входа через ФСП ID: state истёк/подменён, код не принят или ID-токен не прошёл проверку (причина в details.reason)	2
400	token_invalid	Ссылка недействительна	2
401	invalid_api_token	API-токен не найден или отозван	1
401	invalid_credentials	Неверная почта или пароль	1
401	invalid_refresh_token	Сессия истекла, войдите снова	1
401	invalid_signature	Неверная подпись вебхука	1
401	unauthorized	Требуется вход	108
401	webhook_expired	Событие устарело (sent_at старше 5 минут)	1
403	company_blocked	Компания заблокирована администратором платформы	4
403	email_not_verified	Подтвердите почту по ссылке из письма	1
403	forbidden	Недостаточно прав	105
403	no_contact_with_company	Не было контакта с компанией	1
403	user_blocked	Учётная запись заблокирована	2
404	company_invite_not_found	Код приглашения в компанию не найден	1
404	fsp_account_not_linked	ФСП ID не привязан ни к одному профилю	1
404	not_found	Объект не найден	69
409	already_applied	Вы уже откликнулись на эту вакансию	1
409	already_invited	У кандидата уже есть активное приглашение	1
409	bank_empty	Для специализации пока нет задач	1
409	cannot_change_self	Нельзя заблокировать или понизить себя	1
409	consent_required	Согласие на обработку ПДн обязательно; удалите аккаунт	1
409	cooldown_active	Сменить грейд можно не чаще раза в 90 дней	1
409	email_taken	Пользователь с такой почтой уже есть	1
409	fsp_already_linked	К профилю уже привязан другой ФСП ID	1
409	fsp_id_taken	Этот ФСП ID уже привязан к другому профилю	1
409	fsp_not_linked	ФСП ID не привязан	2
409	invalid_transition	Действие недоступно в текущем статусе приглашения	12
409	quota_exceeded	Достигнута квота активных приглашений	1
409	recalibration_running	Пересчёт уже идёт	1
409	report_exists	Жалоба на эту компанию уже на рассмотрении	1
409	role_profile_missing	Нет профиля для этой роли	1
409	session_finished	Тест уже завершён	1
409	session_in_progress	Сначала завершите начатый тест	1
409	session_not_finished	Тест ещё не завершён	1

HTTP	code	Пример сообщения	Методов
409	`skill_exists`	Такая технология уже есть	1
409	`skill_in_use`	Технология используется в профилях или вакансиях	1
409	`specialization_exists`	Такая специализация уже есть	1
409	`specialization_in_use`	Специализация используется	1
409	`submission_expired`	Срок ответа истёк	1
409	`topic_exists`	Такая тема уже есть	1
409	`topic_in_use`	По теме есть шаблоны задач	1
409	`wrong_item`	Ответ не на текущую задачу	1
410	`token_expired`	Срок действия ссылки истёк, запросите новую	2
413	`file_too_large`	Файл больше 5 МБ	2
415	`unsupported_file`	Нужен PDF	1
422	`block_requires_resolve`	Заблокировать компанию можно только при подтверждении	1
422	`invalid_csv`	Неверный формат CSV	1
422	`invalid_kind`	Вид теста не соответствует текущему грейду	1
422	`invalid_thresholds`	Пороги θ должны возрастать	1
422	`invalid_webhook_url`	Адрес должен начинаться с http:// или https://	1
422	`invalid_weights`	Хотя бы один вес больше нуля	1
422	`salary_below_expectations`	Вилка ниже ожиданий кандидата	1
422	`topic_code_invalid`	Код темы — .	1
422	`unknown_category`	Неизвестная категория	1
422	`unknown_skills`	Технологий нет в справочнике стека	3
422	`unknown_specialization`	Нет такой специализации	1
422	`validation_error`	Ошибка валидации данных	60
429	`too_many_attempts`	Слишком много попыток, попробуйте через 15 минут	1
429	`too_many_requests`	Повторить можно через минуту	2
503	`fsp_registry_unavailable`	Реестр ФСП недоступен, данные из кэша	2
503	`keycloak_unavailable`	Сервис ФСП ID недоступен	3
503	`webhook_disabled`	Секрет вебхука не настроен	1

7. Библиотеки и компоненты с версиями

Снято 07.10.2026 с локального dev-стека: версии Python-пакетов — из установленных в контейнерах, версии npm — из `package-lock.json`, образы — из compose-файлов и Dockerfile. Лицензия проекта — MIT (`LICENSE`).

Лицензии. Все компоненты — открытые: MIT, BSD, Apache-2.0, ISC, PSF и аналогичные разрешительные. MPL-2.0 (certifi — корневые сертификаты; lightningcss и caniuse-lite — только при сборке CSS) — слабый копилефт на уровне файла, на наш код не распространяется; шрифты Inter и JetBrains Mono — SIL OFL-1.1. Проприетарных и закрытых компонентов нет. Внешний LLM-провайдер необязателен и подключается только через переменные окружения.

Платформа

Компонент	Версия
Python (бэкенд, мок реестра)	3.12.15 (в `pyproject.toml`: `⩾3.12`)
Node.js (dev-сервер и сборка фронта)	22.23.3 (образ `node:22-alpine`)
PostgreSQL	16.15
Keycloak (ФСП ID)	25.0.6

Библиотеки

Бэкенд (`backend/pyproject.toml`)

Python 3.12.15 (в образе). Прямых зависимостей: 14, всего с транзитивными: 44.

Пакет	Объявлено	Установлено	Лицензия	Зачем
alembic	`alembic⩾1.14`	1.20.0	MIT	миграции схемы БД
asyncpg	`asyncpg⩾0.30`	0.31.0	Apache-2.0	драйвер PostgreSQL
email-validator	`email-validator⩾2.2`	2.3.0	Unlicense	проверка адресов почты
fastapi	`fastapi⩾0.115,<1.0`	0.142.2	MIT	веб-фреймворк API, OpenAPI/Swagger
httpx	`httpx⩾0.27`	0.28.1	BSD-3-Clause	HTTP-клиент: Keycloak, реестр ФСП, LLM
numpy	`numpy⩾2.0`	2.5.3	BSD-3-Clause AND 0BSD AND MIT AND Zlib AND CC0-1.0	движок адаптивного теста (IRT, EAP, калибровка)
pdfplumber	`pdfplumber⩾0.11`	0.11.10	MIT License	разбор PDF-резюме
pydantic	`pydantic⩾2.9,<3`	2.13.5	MIT	схемы запросов и ответов, валидация
pydantic-settings	`pydantic-settings⩾2.6`	2.15.0	MIT	настройки из переменных окружения
PyJWT	`pyjwt[crypto]⩾2.9`	2.15.1	MIT	JWT сессий и проверка ID-токенов Keycloak (JWKS)
python-multipart	`python-multipart⩾0.0.12`	0.0.32	Apache-2.0	загрузка файлов (PDF-резюме)
reportlab	`reportlab⩾4.2`	5.0.1	BSD License	генерация PDF-профиля кандидата
SQLAlchemy	`sqlalchemy[asyncio]⩾2.0.36,<2.1`	2.0.54	MIT	ORM, асинхронный доступ к PostgreSQL
uvicorn	`uvicorn[standard]⩾0.30`	0.54.0	BSD-3-Clause	ASGI-сервер
pytest	`pytest⩾8.3`	только в dev-окружении	—	автотесты
ruff	`ruff⩾0.7`	только в dev-окружении	—	линтер и форматтер
mypy	`mypy⩾1.13`	только в dev-окружении	—	проверка типов

Транзитивные: annotated-doc 0.0.5 (MIT), annotated-types 0.8.0 (MIT), anyio 4.15.1 (MIT), certifi 2026.7.22 (MPL-2.0), cffi 2.1.1 (MIT-0), charset-normalizer 3.5.2 (MIT), click 8.5.0 (BSD-3-Clause), cryptography 50.0.2 (Apache-2.0 OR BSD-3-Clause), dnspython 2.8.0 (ISC), greenlet 3.5.6 (MIT AND PSF-2.0), h11 0.16.0 (MIT), httpcore 1.0.9 (BSD-3-Clause), httptools 0.8.0 (MIT), idna 3.20 (BSD-3-Clause), Mako 1.4.3 (MIT), MarkupSafe 3.0.4 (BSD-3-Clause), opentelemetry-api 1.45.1 (Apache-2.0), pdfminer.six 20260107 (MIT), pillow 12.3.0 (MIT-CMU), pycparser 3.0 (BSD-3-Clause), pydantic_core 2.46.5 (MIT), pypdfium2 5.14.0 (BSD-3-Clause, Apache-2.0, dependency licenses), python-dotenv 1.2.4 (BSD-3-Clause), PyYAML 6.0.3 (MIT), starlette 1.7.0 (BSD-3-Clause), typing_extensions 4.16.0 (PSF-2.0), typing-inspection 0.4.4 (MIT), uvloop 0.23.0 (MIT License), watchfiles 1.3.0 (MIT), websockets 17.2 (BSD-3-Clause).

Мок реестра ФСП (`services/fsp-registry-mock/requirements.txt`)

Python 3.12.15 (в образе). Прямых зависимостей: 3, всего с транзитивными: 14.

Пакет	Объявлено	Установлено	Лицензия	Зачем
fastapi	`fastapi==0.142.2`	0.142.2	MIT	веб-фреймворк API, OpenAPI/Swagger
pydantic	`pydantic==2.13.5`	2.13.5	MIT	схемы запросов и ответов, валидация
uvicorn	`uvicorn==0.54.0`	0.54.0	BSD-3-Clause	ASGI-сервер

Транзитивные: annotated-doc 0.0.5 (MIT), annotated-types 0.8.0 (MIT), anyio 4.15.1 (MIT), click 8.5.0 (BSD-3-Clause), h11 0.16.0 (MIT), idna 3.20 (BSD-3-Clause), opentelemetry-api 1.45.0 (Apache-2.0), pydantic_core 2.46.5 (MIT), starlette 1.7.0 (BSD-3-Clause), typing_extensions 4.16.0 (PSF-2.0), typing-inspection 0.4.4 (MIT).

Фронтенд (`frontend/package.json`, версии из `package-lock.json`)

Попадают в сборку:

Пакет	Объявлено	Установлено	Лицензия	Зачем
@fontsource/inter	`^5.3.0`	5.3.0	OFL-1.1	шрифт Inter (брендбук ФСП), без CDN
@fontsource/jetbrains-mono	`^5.3.0`	5.3.0	OFL-1.1	шрифт JetBrains Mono (брендбук ФСП), без CDN
@tanstack/react-query	`^5.104.1`	5.104.1	MIT	запросы к API и кэш
clsx	`^2.1.1`	2.1.1	MIT	склейка CSS-классов
lucide-react	`^1.52.0`	1.52.0	ISC	иконки
react	`^18.3.1`	18.3.1	MIT	UI
react-dom	`^18.3.1`	18.3.1	MIT	UI (рендер в браузере)
react-router	`^7.18.4`	7.18.4	MIT	маршруты кабинетов
recharts	`^3.10.1`	3.10.1	MIT	графики (валидация, дашборды)
tailwind-merge	`^3.7.0`	3.7.0	MIT	склейка классов Tailwind

Для разработки и сборки:

Пакет	Объявлено	Установлено	Лицензия	Зачем
@eslint/js	`^10.0.1`	10.0.1	MIT	линтер
@tailwindcss/vite	`^4.3.3`	4.3.3	MIT	Tailwind для Vite
@testing-library/dom	`^10.4.2`	10.4.2	MIT	тесты компонентов
@testing-library/jest-dom	`^7.0.1`	7.0.1	MIT	проверки DOM в тестах
@testing-library/react	`^16.3.3`	16.3.3	MIT	тесты компонентов
@testing-library/user-event	`^14.6.7`	14.6.7	MIT	клики и ввод в тестах
@types/node	`^22.20.5`	22.20.5	MIT	типы Node.js
@types/react	`^18.3.31`	18.3.31	MIT	типы React
@types/react-dom	`^18.3.7`	18.3.7	MIT	типы React DOM
@vitejs/plugin-react	`^6.1.2`	6.1.2	MIT	React для Vite
eslint	`^10.12.0`	10.12.0	MIT	линтер
eslint-plugin-react-hooks	`^7.1.1`	7.1.1	MIT	линтер хуков React
eslint-plugin-react-refresh	`^0.5.7`	0.5.7	MIT	линтер Fast Refresh
globals	`^17.13.0`	17.13.0	MIT	линтер (глобальные

Пакет	Объявлено	Установлено	Лицензия	Зачем
				имена)
jsdom	`^30.1.2`	30.1.2	MIT	DOM для тестов
msw	`^2.15.0`	2.15.0	MIT	моки API в браузере (прототип без бэкенда)
openapi-typescript	`^7.13.0`	7.13.0	MIT	TS-типы из OpenAPI
tailwindcss	`^4.3.3`	4.3.3	MIT	стили
typescript	`~5.9.3`	5.9.3	Apache-2.0	типизация
typescript-eslint	`^8.71.1`	8.71.1	MIT	линтер TypeScript
vite	`^8.3.2`	8.3.2	MIT	сборка и dev-сервер
vitest	`^5.0.3`	5.0.3	MIT	автотесты фронтенда

Всего пакетов в lock-файле: 410 (в сборку — 56, только для разработки — 354).

Лицензия	В сборку	Для разработки
MIT	40	269
ISC	12	20
MPL-2.0	0	24
Apache-2.0	0	19
BSD-2-Clause	0	8
BSD-3-Clause	1	3
BlueOak-1.0.0	0	4
(MIT OR CC0-1.0)	0	2
MIT-0	0	2
OFL-1.1	2	0
CC-BY-4.0	0	1
CC0-1.0	0	1
MIT AND ISC	1	0
Python-2.0	0	1

Лицензии вне списка MIT/BSD/Apache/ISC/PSF (проверены вручную, все открытые): @fontsource/inter 5.3.0 (OFL-1.1); @fontsource/jetbrains-mono 5.3.0 (OFL-1.1).

Образы Docker

Образ (тег)	Точная версия	Лицензия	Роль	Где используется
`axllent/mailpit:latest`	1.31.4	MIT	почта в разработке и на демо-стенде	docker-compose.yml, infra/stand/compose.yml
`caddy:2-alpine`	образ не скачан локально	Apache-2.0	HTTPS (Let's Encrypt), обратный прокси и статика фронта на стенде	infra/stand/compose.yml
`node:22-alpine`	22.23.3 (по образу `fsp-talent-frontend`)	MIT	dev-сервер фронта и сборка статики	frontend/Dockerfile, infra/stand/deploy.sh
`postgres:16-alpine`	16.15	PostgreSQL License	СУБД	docker-compose.yml, infra/stand/compose.yml
`python:3.12-slim`	3.12.15 (по образу `fsp-talent-backend`)	PSF-2.0	база образов backend и мока реестра	backend/Dockerfile, services/fsp-registry-mock/Dockerfile
`quay.io/keycloak/keycloak:25.0`	25.0.6	Apache-2.0	ФСП ID: OIDC-провайдер, realm fsp	docker-compose.yml, infra/stand/compose.yml

Собственные образы: `backend` (python:3.12-slim + зависимости бэкенда), `fsp-registry-mock` (python:3.12-slim), `frontend` (node:22-alpine, только для dev). На стенде фронт собирается в статику одноразовым контейнером `node:22-alpine` и раздаётся Caddy.

8. Сборка, развёртывание и локальный запуск

Решение запускается одной командой `docker compose` на любой ОС с Docker. Ниже — три сценария: **локальный запуск полного стека** (для эксперта), **стенд на сервере** (как развёрнут https://159-194-200-243.sslip.io) и **разработка**.

8.1. Что нужно

Что	Версия / объём
Docker Engine или Docker Desktop с Compose v2	Docker 24+, Compose 2.24+ (проверено на Docker Desktop, Compose 5.5)
Память для Docker	4 ГБ минимум, 8 ГБ комфортно (лимиты dev-стека в сумме ≈ 3,3 ГБ)
Диск	≈ 5 ГБ под образы
git	любой
Свободные порты	18000, 15173, 15432, 18025, 11025, 18180, 18100

Интернет нужен только при первой сборке (образы, pip, npm). Внешние API и ключи не нужны: LLM необязателен, реестр ФСП и Keycloak — свои контейнеры.

8.2. Локальный запуск полного стека

```
git clone https://github.com/Vadik145-beep/fsp-talent.git
cd fsp-talent

cp .env.example .env
# в .env поменять одну строку — фронт на реальном API вместо моков:
#   VITE_USE_MOCKS=false

docker compose up -d --build                    # первый запуск — 5-15 минут (сборка образов)
                                                # если упал на «postgres is unhealthy» —
повторить
docker compose exec backend python -m app.seed  # демо-данные: учётки, кандидаты, компании,
приглашения
docker compose ps                               # все сервисы — healthy (Keycloak стартует
~1-2 мин)
```

Бэкенд при старте сам применяет миграции (`alembic upgrade head`) и создаёт администратора. Загрузка демо-данных идемпотентна: повторный запуск ничего не дублирует (`python -m app.seed --reset` — очистить и залить заново). Банк задач (192 шаблона) бэкенд кладёт в БД сам при первом обращении к тесту; явно — `docker compose exec backend python -m app.testing.sync_bank`.

Открыть:

Что	Адрес
Приложение	http://localhost:15173
Swagger API	http://localhost:18000/api/docs
Почта (письма подтверждения, приглашений)	http://localhost:18025
Keycloak (ФСП ID), админка `admin` / `admin`	http://localhost:18180
Мок реестра ФСП, Swagger	http://localhost:18100/docs
PostgreSQL	`localhost:15432`, БД `fsp_talent`, пользователь `fsp` / `fsp`

Демо-учётки (пароль `demo1234`):

Роль	Почта	Что показать
Соискатель	`candidate@example.com`	Backend · Middle, ФСП ID с достижениями, есть приглашения и задание
Соискатель без ФСП	`candidate.nofsp@example.com`	Frontend · Junior, истории ФСП нет — путь без штрафа
Рекрутер	`recruiter@example.com`	компания «Северный код», потребности и приглашения
Администратор	`admin@example.com`	банк задач, детектор утечек, настройки, валидация

Тестовые участники ФСП ID для привязки (в Keycloak): `fsp.user01` … `fsp.user05` (с достижениями), `fsp.nohistory` (без истории), пароль `Fsp-demo-2026`.

Новый пользователь регистрируется на `/register`; письмо со ссылкой подтверждения приходит в Mailpit (http://localhost:18025).

Проверка работоспособности

```
curl http://localhost:18000/api/health
# {"status":"ok","version":"0.1.0","db":"ok","db_revision":"0007_merge_wave5"}

curl -s -H 'Content-Type: application/json' \
  -d '{"email":"recruiter@example.com","password":"demo1234"}' \
  http://localhost:18000/api/auth/login
# {"access_token":"…","token_type":"bearer","expires_in":900,"user":{…,"role":"recruiter"}}

curl http://localhost:18100/health
# {"status":"ok","participants":40,"synthetic":true}
```

Сквозной сценарий в браузере: войти рекрутером → «Потребности» → открыть потребность → подборка с «почему в подборке» → «Пригласить» (вилка, способ связи) → выйти, войти соискателем `candidate@example.com` → «Приглашения» → принять → у рекрутера появились контакты.

Облегчённые варианты

Задача	Команда
Только бэкенд и БД (без фронта и Keycloak)	`docker compose up -d --build postgres backend mailpit fsp-registry-mock`
Только прототип интерфейса на моках, без бэкенда	`cd frontend && npm ci && npm run dev` → http://localhost:15173 (`VITE_USE_MOCKS` не задан = моки)

Остановка

```
docker compose down        # остановить, данные БД сохраняются в томе pgdata
docker compose down -v     # остановить и удалить данные
```

8.3. Переменные окружения

`docker-compose.yml` берёт значения из `.env` в корне (шаблон — `.env.example`); без `.env` работают значения по умолчанию. Основные:

Переменная	По умолчанию	Смысл
`VITE_USE_MOCKS`	`true`	`false` — фронт ходит в настоящий API (Vite проксирует `/api` на backend)
`POSTGRES_USER` / `POSTGRES_PASSWORD` / `POSTGRES_DB`	`fsp` / `fsp` / `fsp_talent`	учётка и база PostgreSQL
`APP_ENV`	`dev`	`dev` / `prod`; вне `dev` refresh-cookie ставится с флагом `Secure` (нужен HTTPS)
`LOG_LEVEL`	`INFO`	уровень логов бэкенда
`JWT_SECRET`	`change-me-in-production`	ключ подписи JWT — на сервере обязательно свой, 32+ символа
`DEMO_AUTH_ENABLED`	`true`	демо-заголовки `X-Demo-Role` вместо JWT; на сервере — `false`
`PUBLIC_FRONTEND_URL`	`http://localhost:15173`	адрес фронта для ссылок в письмах и redirect ФСП ID
`KEYCLOAK_PUBLIC_URL`	`http://localhost:18180`	адрес Keycloak, который видит браузер
`KEYCLOAK_ADMIN` / `KEYCLOAK_ADMIN_PASSWORD`	`admin` / `admin`	админка Keycloak
`LLM_BASE_URL`, `LLM_API_KEY`, `LLM_MODEL`	пусто	OpenAI-совместимый endpoint (`/chat/completions`); пусто — всё на правилах

Задаются в compose и обычно не меняются: `DATABASE_URL`, `SMTP_HOST` / `SMTP_PORT` (Mailpit), `KEYCLOAK_INTERNAL_URL`, `KEYCLOAK_REALM=fsp`, `KEYCLOAK_CLIENT_ID=fsp-talent`, `FSP_REGISTRY_URL`, `SEED_DIR`.

Дополнительные настройки бэкенда — переменные окружения контейнера `backend`, значения по умолчанию — в коде. Dev-compose их в контейнер **не передаёт** (действуют значения по умолчанию); чтобы поменять, добавьте строку в `environment` сервиса `backend`. На стенде часть из них задаётся в `infra/stand/.env`:

Переменная	По умолчанию	Смысл
`SEED_ON_STARTUP`	`false`	загружать демо-данные при старте (на стенде `true`)
`ADMIN_EMAIL` / `ADMIN_PASSWORD`	`admin@example.com` / `demo1234`	администратор, создаётся при старте, если такой почты ещё нет; локально всегда `demo1234` (строка `ADMIN_PASSWORD` в корневом `.env` в контейнер не попадает), на стенде — из `infra/stand/.env`
`CORS_ORIGINS`	`["http://localhost:15173","http://127.0.0.1:15173"]`	разрешённые origin фронта (JSON-список)
`ACCESS_TOKEN_TTL_MIN` / `REFRESH_TOKEN_TTL_DAYS`	15 / 30	срок жизни токенов
`LOGIN_MAX_ATTEMPTS` / `LOGIN_WINDOW_MIN`	5 / 15	лимит неудачных входов
`SMTP_USER`, `SMTP_PASSWORD`, `SMTP_STARTTLS`, `SMTP_FROM`	пусто / `false` / `noreply@fsp-talent.example.com`	настоящий SMTP вместо Mailpit
`FSP_WEBHOOK_SECRET`	`dev-fsp-webhook-secret`	секрет HMAC вебхука реестра; пусто — вебхук выключен
`FSP_REGISTRY_TOKEN`	пусто	сервисный токен для реестра
`TESTING_CAT_SE_STOP` / `TESTING_CAT_MAX_ITEMS` / `TESTING_CAT_MIN_ITEMS`	0.30 / 20 / 6	правило остановки теста
`TESTING_LEAK_SCAN_EVERY`	50	детектор утечек после каждой N-й сессии
`RESUME_MAX_BYTES` / `RESUME_MAX_PAGES`	5 МБ / 10	ограничения PDF-резюме
`TASKS_FRESHNESS_ANSWER_BONUS` / `TASKS_FRESHNESS_GRADE_BASE` / `TASKS_FRESHNESS_GRADE_PER_POINT`	0.08 / 0.04 / 0.03	прибавки свежести профиля за ответ и за оценку 0–3
`TASKS_FRESHNESS_HALF_LIFE_DAYS` / `TASKS_MAX_ASSIGN_PER_RUN`	30 / 50	полураспад свежести; сколько кандидатов получают задание за проход
`INTEGRATIONS_WEBHOOK_MAX_ATTEMPTS` / `INTEGRATIONS_WEBHOOK_TIMEOUT_SEC`	5 / 5	попытки доставки вебхука ATS и таймаут запроса
`INTEGRATIONS_ALLOW_PRIVATE_URLS`	`true`	разрешить адреса localhost и частных сетей в вебхуках; на проде — `false` (защита от SSRF)

Пороги θ, кулдауны, квота и срок приглашений, веса ранжирования — не переменные, а настройки платформы в БД: их меняет администратор на экране «Справочники».

8.4. Стенд на сервере (один сервер, Caddy + HTTPS)

Так развёрнут https://159-194-200-243.sslip.io. Файлы — `infra/stand/`: `compose.yml` (Caddy, backend, PostgreSQL, Keycloak, мок реестра, Mailpit), `Caddyfile`, `.env.example`, `deploy.sh`. Сервер: 2 vCPU / 4 ГБ, Ubuntu с Docker, открыты порты 80 и 443. Домен — бесплатный `<ip-через-дефисы>.sslip.io`, сертификат Let's Encrypt Caddy получает сам.

Один адрес обслуживает всё:

Путь	Куда
`/`	статика фронта (`infra/stand/site`, собранная с `VITE_USE_MOCKS=false`)
`/api/*`	backend
`/kc/*`	Keycloak (`KC_HTTP_RELATIVE_PATH=/kc`)
`/mail*`	Mailpit — демо-почта для писем подтверждения

Первый запуск:

```
git clone https://github.com/Vadik145-beep/fsp-talent.git /opt/fsp-talent/app
cd /opt/fsp-talent/app/infra/stand
cp .env.example .env
```

```
# в .env: STAND_HOST=<ваш-ip-через-дефисы>.sslip.io и случайные секреты:
#   POSTGRES_PASSWORD, JWT_SECRET (32+ символа), KEYCLOAK_ADMIN_PASSWORD, FSP_WEBHOOK_SECRET
#   (например: openssl rand -hex 24); ADMIN_PASSWORD — пароль администратора
chmod 600 .env
./deploy.sh
```

`deploy.sh` делает: `git reset --hard origin/main` → сборку фронта в одноразовом контейнере `node:22-alpine` с `VITE_USE_MOCKS=false` → копирование `frontend/dist` в `infra/stand/site` → `docker compose --env-file .env up -d --build` → `python -m app.testing.sync_bank`. Повторный запуск — обновление стенда до свежего `main`. Скрипт рассчитан на путь `/opt/fsp-talent/app` (переменная `APP` в начале файла).

Отличия стенда от dev-стека: `APP_ENV=prod` (secure-cookie), `DEMO_AUTH_ENABLED=false` (только JWT), `SEED_ON_STARTUP=true` (демо-данные загружаются сами), фронт — собранная статика, а не Vite; наружу открыт только Caddy.

Проверка стенда:

```
curl https://<STAND_HOST>/api/health
docker compose -f infra/stand/compose.yml --env-file infra/stand/.env ps
docker compose -f infra/stand/compose.yml --env-file infra/stand/.env logs -f backend
```

Резервная копия БД: `docker compose --env-file .env exec -T postgres pg_dump -U fsp fsp_talent > backup.sql`.

8.5. Разработка

Бэкенд без Docker (Python 3.12+), БД — из compose:

```
docker compose up -d postgres
cd backend
pip install -e ".[dev]"
export DATABASE_URL=postgresql+asyncpg://fsp:fsp@localhost:15432/fsp_talent
alembic upgrade head
python -m app.seed
uvicorn app.main:app --reload --port 18000
```

Проверки (то же делает CI, `.github/workflows/ci.yml`):

```
make check      # ruff + mypy + pytest + актуальность docs/api/openapi.json
cd frontend && npm ci && npm run lint && npm run typecheck && npm test -- --run && npm run build
```

Полезное: `make openapi` — выгрузить схему API; `cd frontend && npm run gen:api` — TS-типы из схемы; `make fixtures` — перегенерировать `data/seed`; `make validate` — полная валидация (данные, симуляции, графики, `docs/validation/report.md`; нужен `matplotlib`); симуляция движка теста — `PYTHONPATH=backend python -m app.testing.engine.simulate --seed 20261011`.

8.6. Периодические задачи (cron)

Бэкенд не держит фоновых потоков: периодические работы — отдельные команды, которые запускаются в контейнере `backend` (их же можно вызвать без cron — см. последнюю строку).

Команда	Что делает	Как часто
`python -m app.tasks daily`	просрочить регулярные задания без ответа, снизить свежесть профилей (полураспад 30 дней)	раз в сутки
`python -m app.tasks distribute`	раздать активные задания кандидатам без активного задания	раз в неделю (или раз в N дней)
`python -m app.integrations`	повторно отправить вебхуки ATS, у которых подошло время повтора	раз в минуту
`POST /api/tasks/jobs/run?job=daily` (или `distribute`, `tick`)	то же для заданий по запросу администратора	вручную

Пример crontab на сервере стенда (`crontab -e` пользователя с доступом к Docker):

```
C="docker compose -f /opt/fsp-talent/app/infra/stand/compose.yml --env-file
/opt/fsp-talent/app/infra/stand/.env exec -T backend"
15 3 * * *  $C python -m app.tasks daily
```

```
0 9 * * 1   $C python -m app.tasks distribute
* * * * *   $C python -m app.integrations
```

Локально (dev-стек) — `docker compose exec backend python -m app.tasks daily` и т.д. Повторный или пропущенный запуск безопасен: снижение свежести считается по времени с прошлого запуска, раздача не выдаёт задание дважды, доставка вебхука не повторяется после успеха. Compose-файл стенда cron не поднимает — расписание ставится на хосте, как показано выше.

8.7. Частые проблемы

Симптом	Причина и решение
`port is already allocated`	порт занят другим процессом — освободите его или поменяйте левую часть `ports` в `docker-compose.yml`
Фронт показывает данные, но вход демо-учёткой «не настоящий», в шапке плашка «Смотреть как»	фронт на моках: в `.env` нет `VITE_USE_MOCKS=false`; после правки — `docker compose up -d frontend` (пересоздать контейнер)
Вход демо-учёткой работает, но кабинеты пустые (нет кандидатов, потребностей, приглашений)	не загружены демо-данные: `docker compose exec backend python -m app.seed` (в dev вход демо-учёток работает и до загрузки — за счёт режима `DEMO_AUTH_ENABLED`)
Первый `docker compose up` завершился ошибкой `dependency failed to start: container …-postgres-1 is unhealthy`	на медленном диске первичная инициализация PostgreSQL дольше таймаута healthcheck; подождите минуту и повторите `docker compose up -d` — данные уже на месте
Новый пользователь не может войти: `403 email_not_verified`	подтвердите почту по ссылке из письма в Mailpit (http://localhost:18025)
Тест не стартует: `409 bank_empty`	банк задач не синхронизирован: `docker compose exec backend python -m app.testing.sync_bank`
Keycloak долго `starting`, вход через ФСП ID отвечает 503	Keycloak стартует 1–2 минуты и требует до 1 ГБ памяти; подождите или увеличьте память Docker. Основной вход работает без Keycloak
После перезагрузки страницы на сервере без HTTPS сессия теряется	при `APP_ENV=prod` refresh-cookie только для HTTPS; локально используйте `APP_ENV=dev` или стенд с Caddy
Фронт на другом адресе получает ошибку CORS	добавьте адрес в `CORS_ORIGINS` (JSON-список)
Не хватает памяти на маленьком сервере	лимиты заданы `mem_limit` в compose; на 4 ГБ включите swap 2 ГБ